CALL OF ACARISTAN

SOURCE CODE ARCHIVAL MONOLITH

GIT METADATA UNAVAILABLE

DATE: 2026-07-19

FILES: 32

TOTAL LINES: 59,098

DENSITY: EXTREME

GENERATED BY BUILD_CODE_PDF.PY

```
...
```

```
# Generated by Django 4.2.11 on 2024-04-22 04:47

from decimal import Decimal
import django.core.validators
import django.db.models.deletion
import django.utils.timezone
import uuid


class Migration(migrations.Migration):

    initial = True

    dependencies = [
        ('auth', '0012_alter_user_first_name_max_length'),
        ('contenttypes', '0002_remove_content_type_name'),
    ]

    operations = [
        migrations.CreateModel(
            name='Account',
            fields=[
                ('id', models.UUIDField(default=uuid.uuid4, editable=False, primary_key=True, serialize=False)),
                ('name', models.CharField(max_length=100)),
                ('account_type', models.CharField(choices=[('CHECKING', 'Checking'), ('SAVINGS', 'Savings'), ('CREDIT_CARD', 'Credit Card'), ('CASH', 'Cash'), ('INVESTMENT', 'Investment'), ('LOAN', 'Loan'), ('OTHER', 'Other')], max_length=20)),
                ('currency', models.CharField(default='USD', max_length=3)),
                ('initial_balance', models.DecimalField(decimal_places=2, default=Decimal('0.00'), max_digits=12)),
                ('current_balance', models.DecimalField(decimal_places=2, default=Decimal('0.00'), max_digits=12)),
                ('is_active', models.BooleanField(default=True)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('updated_at', models.DateTimeField(auto_now=True)),
                ('user', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='accounts', to='auth.user')),
            ],
            options={
                'ordering': ['name'],
            },
        ),
        migrations.CreateModel(
            name='Category',
            fields=[
                ('id', models.UUIDField(default=uuid.uuid4, editable=False, primary_key=True, serialize=False)),
                ('name', models.CharField(max_length=100)),
                ('category_type', models.CharField(choices=[('INCOME', 'Income'), ('EXPENSE', 'Expense'), ('TRANSFER', 'Transfer')], max_length=20)),
                ('icon', models.CharField(blank=True, max_length=50)),
                ('color', models.CharField(blank=True, max_length=7)),
                ('is_system', models.BooleanField(default=False)),
                ('parent', models.ForeignKey(blank=True, null=True, on_delete=django.db.models.deletion.CASCADE, related_name='children', to='finance.category')),
                ('user', models.ForeignKey(blank=True, null=True, on_delete=django.db.models.deletion.CASCADE, related_name='categories', to='auth.user')),
            ],
            options={
                'verbose_name_plural': 'Categories',
                'ordering': ['name'],
            },
        ),
        migrations.CreateModel(
            name='Transaction',
            fields=[
                ('id', models.UUIDField(default=uuid.uuid4, editable=False, primary_key=True, serialize=False)),
                ('amount', models.DecimalField(decimal_places=2, max_digits=12)),
                ('transaction_type', models.CharField(choices=[('INCOME', 'Income'), ('EXPENSE', 'Expense'), ('TRANSFER', 'Transfer')], max_length=20)),
                ('date', models.DateField(default=django.utils.timezone.now)),
                ('description', models.CharField(blank=True, max_length=255)),
                ('notes', models.TextField(blank=True)),
                ('is_recurring', models.BooleanField(default=False)),
                ('recurring_interval', models.CharField(blank=True, choices=[('DAILY', 'Daily'), ('WEEKLY', 'Weekly'), ('BIWEEKLY', 'Bi-weekly'), ('MONTHLY', 'Monthly'), ('YEARLY', 'Yearly')], max_length=20)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('updated_at', models.DateTimeField(auto_now=True)),
                ('account', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='transactions', to='finance.account')),
                ('category', models.ForeignKey(blank=True, null=True, on_delete=django.db.models.deletion.SET_NULL, related_name='transactions', to='finance.category')),
                ('destination_account', models.ForeignKey(blank=True, null=True, on_delete=django.db.models.deletion.SET_NULL, related_name='incoming_transfers', to='finance.account')),
                ('user', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='transactions', to='auth.user')),
            ],
            options={
                'ordering': ['-date', '-created_at'],
            },
        ),
        migrations.CreateModel(
            name='Budget',
            fields=[
                ('id', models.UUIDField(default=uuid.uuid4, editable=False, primary_key=True, serialize=False)),
                ('amount', models.DecimalField(decimal_places=2, max_digits=12)),
                ('period', models.CharField(choices=[('WEEKLY', 'Weekly'), ('MONTHLY', 'Monthly'), ('YEARLY', 'Yearly')], default='MONTHLY', max_length=20)),
                ('start_date', models.DateField()),
                ('end_date', models.DateField(blank=True, null=True)),
                ('is_active', models.BooleanField(default=True)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('category', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='budgets', to='finance.category')),
                ('user', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='budgets', to='auth.user')),
            ],
            options={
                'ordering': ['-start_date'],
            },
        ),
        migrations.CreateModel(
            name='SavingsGoal',
            fields=[
                ('id', models.UUIDField(default=uuid.uuid4, editable=False, primary_key=True, serialize=False)),
                ('name', models.CharField(max_length=100)),
                ('target_amount', models.DecimalField(decimal_places=2, max_digits=12)),
                ('current_amount', models.DecimalField(decimal_places=2, default=Decimal('0.00'), max_digits=12)),
                ('deadline', models.DateField(blank=True, null=True)),
                ('is_completed', models.BooleanField(default=False)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('user', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='savings_goals', to='auth.user')),
            ],
            options={
                'ordering': ['deadline'],
            },
        ),
    ]
```

```
fetch_logged_in_user_app_role_assignments>>>
```

```
def _make_context_manager_invoker(method_name):
    def invoker(self, *args, **kwargs):
        return getattr(self._obj, method_name)(*args, **kwargs)
    return invoker
def _get_connection_info(conn):
    info = conn.info
    return (info.host, info.port, info.user, info.password,
            info.database, info.socket)
def _handle_sql_exception(func, return_value=None, exception_type=None):
    def decorated(*args, **kwargs):
        try:
            return func(*args, **kwargs)
        except exception_type, e:
            if not hasattr(e, '_pydispatch_args'):
                e._pydispatch_args = args
            raise
    return decorated
def _closed_property(name, doc=None):
    def get(self):
        return getattr(self, name) and not self.open
    return property(get, doc=doc)
def _deferred_connect(__connect, exception_handler):
    def connect(self):
        try:
            return __connect(self)
        except exception_handler, e:
            self.failed(e)
    return connect
def _support_segments(segments, segment_type, segment_count):
    if segments is None:
        segments = [segment_type() for x in xrange(segment_count)]
    return segments
def _load_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.load()
    return segments
def _dump_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.dump()
    return segments
def _cache_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.cache()
    return segments
def _load_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.load_segments()
    return segments
def _dump_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.dump_segments()
    return segments
def _cache_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.cache_segments()
    return segments
def _get_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.get_segments()
    return segments
def _set_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.set_segments()
    return segments
def _segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments_segmented_segments(segments, segment_type, segment_count):
    segments = _support_segments(segments, segment_type, segment_count)
    for segment in segments:
        segment.segments()
    return segments
def segments_segmented_
```

```
```

```
```

```
```

```
systemctl list-jobs
```

```
  1 graphical.target                start
  2 multi-user.target               start
  3 network.target                  start
  4 network-online.target           start
  5 nss-lookup.target               start
  6 remote-fs.target                start
  7 sshd.service                    start
  8 systemd-user-sessions.service   start
  9 time-sync.target                start
 10 getty@tty1.service              start
 11 plymouth-quit.service           start
 12 plymouth-quit-wait.service      start
 13 systemd-update-utmp-runlevel.service start
 14 cloud-init.service              start
 15 cloud-init-local.service        start
 16 cloud-config.service            start
 17 cloud-final.service             start
 18 systemd-logind.service          start
 19 systemd-networkd.service        start
 20 systemd-networkd-wait-online.service start
 21 systemd-resolved.service        start
 22 systemd-timesyncd.service       start
 23 systemd-udevd.service           start
 24 systemd-udev-trigger.service    start
 25 systemd-udev-settle.service     start
 26 systemd-journald.service        start
 27 systemd-journal-flush.service   start
 28 systemd-random-seed.service     start
 29 systemd-sysctl.service          start
 30 systemd-modules-load.service    start
 31 systemd-tmpfiles-setup.service  start
 32 systemd-tmpfiles-clean.service  start
 33 systemd-tmpfiles-clean.timer    start
 34 systemd-update-utmp.service     start
 35 systemd-update-utmp-runlevel.service start
 36 systemd-update-utmp-shutdown.service start
 37 systemd-update-utmp-boot.service start
 38 systemd-update-utmp-runlevel.service start
 39 systemd-update-utmp-shutdown.service start
 40 systemd-update-utmp-boot.service start
 41 systemd-update-utmp-runlevel.service start
 42 systemd-update-utmp-shutdown.service start
 43 systemd-update-utmp-boot.service start
 44 systemd-update-utmp-runlevel.service start
 45 systemd-update-utmp-shutdown.service start
 46 systemd-update-utmp-boot.service start
```

```
```

```
```

```
```

```
```

```
    interface Storage extends Remote, Serializable {
        String getName() throws RemoteException;
        void setName(String name) throws RemoteException;
        long getCapacity() throws RemoteException;
        void setCapacity(long capacity) throws RemoteException;
        long getFreeSpace() throws RemoteException;
        void setFreeSpace(long freeSpace) throws RemoteException;
        String getType() throws RemoteException;
        void setType(String type) throws RemoteException;
        String getPath() throws RemoteException;
        void setPath(String path) throws RemoteException;
        boolean isReadOnly() throws RemoteException;
        void setReadOnly(boolean readOnly) throws RemoteException;
        boolean isRemovable() throws RemoteException;
        void setRemovable(boolean removable) throws RemoteException;
        boolean isMounted() throws RemoteException;
        void setMounted(boolean mounted) throws RemoteException;
        String getMountPoint() throws RemoteException;
        void setMountPoint(String mountPoint) throws RemoteException;
        String getFileSystem() throws RemoteException;
        void setFileSystem(String fileSystem) throws RemoteException;
        String getDevice() throws RemoteException;
        void setDevice(String device) throws RemoteException;
        String getVendor() throws RemoteException;
        void setVendor(String vendor) throws RemoteException;
        String getModel() throws RemoteException;
        void setModel(String model) throws RemoteException;
        String getSerialNumber() throws RemoteException;
        void setSerialNumber(String serialNumber) throws RemoteException;
        String getFirmwareVersion() throws RemoteException;
        void setFirmwareVersion(String firmwareVersion) throws RemoteException;
        String getHardwareVersion() throws RemoteException;
        void setHardwareVersion(String hardwareVersion) throws RemoteException;
        String getSoftwareVersion() throws RemoteException;
        void setSoftwareVersion(String softwareVersion) throws RemoteException;
        String getManufacturer() throws RemoteException;
        void setManufacturer(String manufacturer) throws RemoteException;
        String getProductName() throws RemoteException;
        void setProductName(String productName) throws RemoteException;
        String getProductVersion() throws RemoteException;
        void setProductVersion(String productVersion) throws RemoteException;
        String getProductSerialNumber() throws RemoteException;
        void setProductSerialNumber(String productSerialNumber) throws RemoteException;
        String getProductManufacturer() throws RemoteException;
        void setProductManufacturer(String productManufacturer) throws RemoteException;
        String getProductModel() throws RemoteException;
        void setProductModel(String productModel) throws RemoteException;
        String getProductFirmwareVersion() throws RemoteException;
        void setProductFirmwareVersion(String productFirmwareVersion) throws RemoteException;
        String getProductHardwareVersion() throws RemoteException;
        void setProductHardwareVersion(String productHardwareVersion) throws RemoteException;
        String getProductSoftwareVersion() throws RemoteException;
        void setProductSoftwareVersion(String productSoftwareVersion) throws RemoteException;
        String getProductVendor() throws RemoteException;
        void setProductVendor(String productVendor) throws RemoteException;
        String getProductDevice() throws RemoteException;
        void setProductDevice(String productDevice) throws RemoteException;
        String getProductFileSystem() throws RemoteException;
        void setProductFileSystem(String productFileSystem) throws RemoteException;
        String getProductMountPoint() throws RemoteException;
        void setProductMountPoint(String productMountPoint) throws RemoteException;
        String getProductPath() throws RemoteException;
        void setProductPath(String productPath) throws RemoteException;
        String getProductType() throws RemoteException;
        void setProductType(String productType) throws RemoteException;
        String getProductName() throws RemoteException;
        void setProductName(String productName) throws RemoteException;
        String getProductVersion() throws RemoteException;
        void setProductVersion(String productVersion) throws RemoteException;
        String getProductSerialNumber() throws RemoteException;
        void setProductSerialNumber(String productSerialNumber) throws RemoteException;
        String getProductManufacturer() throws RemoteException;
        void setProductManufacturer(String productManufacturer) throws RemoteException;
        String getProductModel() throws RemoteException;
        void setProductModel(String productModel) throws RemoteException;
        String getProductFirmwareVersion() throws RemoteException;
        void setProductFirmwareVersion(String productFirmwareVersion) throws RemoteException;
        String getProductHardwareVersion() throws RemoteException;
        void setProductHardwareVersion(String productHardwareVersion) throws RemoteException;
        String getProductSoftwareVersion() throws RemoteException;
        void setProductSoftwareVersion(String productSoftwareVersion) throws RemoteException;
        String getProductVendor() throws RemoteException;
        void setProductVendor(String productVendor) throws RemoteException;
        String getProductDevice() throws RemoteException;
        void setProductDevice(String productDevice) throws RemoteException;
        String getProductFileSystem() throws RemoteException;
        void setProductFileSystem(String productFileSystem) throws RemoteException;
        String getProductMountPoint() throws RemoteException;
        void setProductMountPoint(String productMountPoint) throws RemoteException;
        String getProductPath() throws RemoteException;
        void setProductPath(String productPath) throws RemoteException;
        String getProductType() throws RemoteException;
        void setProductType(String productType) throws RemoteException;
    }
```

```
    interface Storage extends Remote, Serializable {
        String getName() throws RemoteException;
        void setName(String name) throws RemoteException;
        long getCapacity() throws RemoteException;
        void setCapacity(long capacity) throws RemoteException;
        long getFreeSpace() throws RemoteException;
        void setFreeSpace(long freeSpace) throws RemoteException;
        String getType() throws RemoteException;
        void setType(String type) throws RemoteException;
        String getPath() throws RemoteException;
        void setPath(String path) throws RemoteException;
        boolean isReadOnly() throws RemoteException;
        void setReadOnly(boolean readOnly) throws RemoteException;
        boolean isRemovable() throws RemoteException;
        void setRemovable(boolean removable) throws RemoteException;
        boolean isMounted() throws RemoteException;
        void setMounted(boolean mounted) throws RemoteException;
        String getMountPoint() throws RemoteException;
        void setMountPoint(String mountPoint) throws RemoteException;
        String getFileSystem() throws RemoteException;
        void setFileSystem(String fileSystem) throws RemoteException;
        String getDevice() throws RemoteException;
        void setDevice(String device) throws RemoteException;
        String getVendor() throws RemoteException;
        void setVendor(String vendor) throws RemoteException;
        String getModel() throws RemoteException;
        void setModel(String model) throws RemoteException;
        String getSerialNumber() throws RemoteException;
        void setSerialNumber(String serialNumber) throws RemoteException;
        String getFirmwareVersion() throws RemoteException;
        void setFirmwareVersion(String firmwareVersion) throws RemoteException;
        String getHardwareVersion() throws RemoteException;
        void setHardwareVersion(String hardwareVersion) throws RemoteException;
        String getSoftwareVersion() throws RemoteException;
        void setSoftwareVersion(String softwareVersion) throws RemoteException;
        String getManufacturer() throws RemoteException;
        void setManufacturer(String manufacturer
```

```
```

```
```

```
# Configuration generated by dpkg-reconfigure
# This system is using the eCos kernel
# Automatically generated make config: don't edit
# Linux kernel version: 2.6.39-rc1
# Wed Apr 20 20:03:54 2011
CONFIG_ARM=y
CONFIG_SYS_SUPPORTS_APM_EMULATION=y
CONFIG_GENERIC_GPIO=y
CONFIG_GENERIC_TIME=y
CONFIG_GENERIC_CLOCKEVENTS=y
CONFIG_GENERIC_CLOCKEVENTS_BROADCAST=y
CONFIG_HAVE_TCM=y
CONFIG_GENERIC_HARDIRQS=y
CONFIG_STACKTRACE_SUPPORT=y
CONFIG_HAVE_LATENCYTOP_SUPPORT=y
CONFIG_LOCKDEP_SUPPORT=y
CONFIG_TRACE_IRQFLAGS_SUPPORT=y
CONFIG_HARDIRQS_SW_RESEND=y
CONFIG_GENERIC_IRQ_PROBE=y
CONFIG_RWSEM_GENERIC_SPINLOCK=y
# CONFIG_ARCH_HAS_ILOG2_U32 is not set
# CONFIG_ARCH_HAS_ILOG2_U64 is not set
CONFIG_GENERIC_HWEIGHT=y
CONFIG_GENERIC_CALIBRATE_DELAY=y
CONFIG_ARCH_MAY_HAVE_PC_FDC=y
CONFIG_ZONE_DMA=y
CONFIG_NEED_DMA_MAP_STATE=y
CONFIG_VECTORS_BASE=0xffff0000
CONFIG_ARM_PATCH_PHYS_VIRT=y
CONFIG_GENERIC_BUG=y
CONFIG_DEFCONFIG_LIST="/lib/modules/$UNAME_RELEASE/.config"
CONFIG_IRQ_WORK=y
CONFIG_BUILDTIME_EXTABLE_SORT=y
CONFIG_CONSTRUCTORS=y
CONFIG_HAVE_KERNEL_GZIP=y
CONFIG_HAVE_KERNEL_LZMA=y
CONFIG_HAVE_KERNEL_LZO=y
CONFIG_KERNEL_GZIP=y
# CONFIG_KERNEL_LZMA is not set
# CONFIG_KERNEL_LZO is not set
CONFIG_DEFAULT_HOSTNAME="(none)"
CONFIG_SWAP=y
CONFIG_SYSVIPC=y
CONFIG_SYSVIPC_SYSCTL=y
CONFIG_POSIX_MQUEUE=y
CONFIG_POSIX_MQUEUE_SYSCTL=y
CONFIG_BSD_PROCESS_ACCT=y
# CONFIG_BSD_PROCESS_ACCT_V3 is not set
CONFIG_FHANDLE=y
CONFIG_TASKSTATS=y
CONFIG_TASK_DELAY_ACCT=y
CONFIG_TASK_XACCT=y
CONFIG_TASK_IO_ACCOUNTING=y
CONFIG_AUDIT=y
CONFIG_AUDITSYSCALL=y
CONFIG_AUDIT_WATCH=y
CONFIG_AUDIT_TREE=y
CONFIG_AUDIT_LOGINUID_IMMUTABLE=y
CONFIG_HAVE_GENERIC_HARDIRQS=y
# IRQ subsystem
CONFIG_GENERIC_HARDIRQS=y
CONFIG_HAVE_SPARSE_IRQ=y
CONFIG_GENERIC_IRQ_SHOW=y
CONFIG_GENERIC_IRQ_CHIP=y
CONFIG_IRQ_DOMAIN=y
CONFIG_IRQ_FORCED_THREADING=y
CONFIG_SPARSE_IRQ=y
# RCU Subsystem
CONFIG_TREE_PREEMPT_RCU=y
CONFIG_PREEMPT_RCU=y
CONFIG_RCU_FANOUT=32
# CONFIG_RCU_FANOUT_EXACT is not set
# CONFIG_RCU_FAST_NO_HZ is not set
# CONFIG_TREE_RCU_TRACE is not set
CONFIG_IKCONFIG=y
CONFIG_IKCONFIG_PROC=y
CONFIG_LOG_BUF_SHIFT=14
CONFIG_CGROUPS=y
# CONFIG_CGROUP_DEBUG is not set
CONFIG_CGROUP_FREEZER=y
CONFIG_CGROUP_DEVICE=y
CONFIG_CPUSETS=y
CONFIG_PROC_PID_CPUSET=y
CONFIG_CGROUP_CPUACCT=y
CONFIG_RESOURCE_COUNTERS=y
CONFIG_MEMCG=y
CONFIG_MEMCG_SWAP=y
CONFIG_MEMCG_SWAP_ENABLED=y
CONFIG_MEMCG_KMEM=y
CONFIG_CGROUP_PERF=y
CONFIG_CGROUP_SCHED=y
CONFIG_FAIR_GROUP_SCHED=y
# CONFIG_CFS_BANDWIDTH is not set
CONFIG_RT_GROUP_SCHED=y
CONFIG_BLK_CGROUP=y
# CONFIG_DEBUG_BLK_CGROUP is not set
CONFIG_NAMESPACES=y
CONFIG_UTS_NS=y
CONFIG_IPC_NS=y
CONFIG_USER_NS=y
CONFIG_PID_NS=y
CONFIG_NET_NS=y
CONFIG_SCHED_AUTOGROUP=y
# CONFIG_SYSFS_DEPRECATED is not set
CONFIG_RELAY=y
CONFIG_BLK_DEV_INITRD=y
CONFIG_INITRAMFS_SOURCE=""
CONFIG_RD_GZIP=y
CONFIG_RD_BZIP2=y
CONFIG_RD_LZMA=y
CONFIG_RD_XZ=y
CONFIG_RD_LZO=y
CONFIG_CC_OPTIMIZE_FOR_SIZE=y
CONFIG_SYSCTL=y
CONFIG_ANON_INODES=y
CONFIG_EXPERT=y
CONFIG_UID16=y
CONFIG_SYSCTL_SYSCALL=y
CONFIG_KALLSYMS=y
CONFIG_KALLSYMS_ALL=y
CONFIG_HOTPLUG=y
CONFIG_PRINTK=y
CONFIG_BUG=y
CONFIG_ELF_CORE=y
CONFIG_BASE_FULL=y
CONFIG_FUTEX=y
CONFIG_EPOLL=y
CONFIG_SIGNALFD=y
CONFIG_TIMERFD=y
CONFIG_EVENTFD=y
CONFIG_SHMEM=y
CONFIG_AIO=y
CONFIG_EMBEDDED=y
CONFIG_HAVE_PERF_EVENTS=y
CONFIG_PERF_USE_VMALLOC=y
# Kernel Performance Events And Counters
CONFIG_PERF_EVENTS=y
# CONFIG_PERF_COUNTERS is not set
# CONFIG_DEBUG_PERF_USE_VMALLOC is not set
CONFIG_VM_EVENT_COUNTERS=y
CONFIG_PCI_QUIRKS=y
CONFIG_SLUB_DEBUG=y
# CONFIG_COMPAT_BRK is not set
# CONFIG_SLAB is not set
CONFIG_SLUB=y
CONFIG_PROFILING=y
CONFIG_TRACEPOINTS=y
CONFIG_OPROFILE=y
CONFIG_HAVE_OPROFILE=y
CONFIG_OPROFILE_NMI_TIMER=y
CONFIG_KPROBES=y
CONFIG_KRETPROBES=y
CONFIG_HAVE_KPROBES=y
CONFIG_HAVE_KRETPROBES=y
CONFIG_HAVE_ARCH_TRACEHOOK=y
CONFIG_HAVE_DMA_ATTRS=y
CONFIG_USE_GENERIC_SMP_HELPERS=y
CONFIG_HAVE_REGS_AND_STACK_ACCESS_API=y
CONFIG_HAVE_CLK=y
CONFIG_HAVE_DMA_API_DEBUG=y
CONFIG_HAVE_HW_BREAKPOINT=y
CONFIG_HAVE_ARCH_JUMP_LABEL=y
CONFIG_HAVE_DMA_CONTIGUOUS=y
CONFIG_HAVE_ARCH_KGDB=y
CONFIG_HAVE_ARCH_TRANSPARENT_HUGEPAGE=y
CONFIG_HAVE_MEMBLOCK=y
CONFIG_ARCH_DISCARD_MEMBLOCK=y
CONFIG_NO_BOOTMEM=y
CONFIG_HAVE_GENERIC_DMA_COHERENT=y
CONFIG_SLABINFO=y
CONFIG_RT_MUTEXES=y
CONFIG_BASE_SMALL=0
CONFIG_MODULES=y
CONFIG_MODULE_FORCE_LOAD=y
CONFIG_MODULE_UNLOAD=y
CONFIG_MODULE_FORCE_UNLOAD=y
CONFIG_MODVERSIONS=y
CONFIG_MODULE_SRCVERSION_ALL=y
CONFIG_STOP_MACHINE=y
CONFIG_BLOCK=y
CONFIG_LBDAF=y
CONFIG_BLK_DEV_BSG=y
CONFIG_BLK_DEV_BSGLIB=y
CONFIG_BLK_DEV_INTEGRITY=y
# IO Schedulers
CONFIG_IOSCHED_NOOP=y
CONFIG_IOSCHED_DEADLINE=y
CONFIG_IOSCHED_CFQ=y
CONFIG_CFQ_GROUP_IOSCHED=y
# CONFIG_DEFAULT_DEADLINE is not set
CONFIG_DEFAULT_CFQ=y
# CONFIG_DEFAULT_NOOP is not set
CONFIG_DEFAULT_IOSCHED="cfq"
CONFIG_INLINE_SPIN_TRYLOCK=y
CONFIG_INLINE_SPIN_TRYLOCK_BH=y
CONFIG_INLINE_SPIN_LOCK=y
CONFIG_INLINE_SPIN_LOCK_BH=y
CONFIG_INLINE_SPIN_LOCK_IRQ=y
CONFIG_INLINE_SPIN_LOCK_IRQSAVE=y
CONFIG_INLINE_SPIN_UNLOCK=y
CONFIG_INLINE_SPIN_UNLOCK_BH=y
CONFIG_INLINE_SPIN_UNLOCK_IRQ=y
CONFIG_INLINE_SPIN_UNLOCK_IRQRESTORE=y
CONFIG_INLINE_READ_TRYLOCK=y
CONFIG_INLINE_READ_LOCK=y
CONFIG_INLINE_READ_LOCK_BH=y
CONFIG_INLINE_READ_LOCK_IRQ=y
CONFIG_INLINE_READ_LOCK_IRQSAVE=y
CONFIG_INLINE_READ_UNLOCK=y
CONFIG_INLINE_READ_UNLOCK_BH=y
CONFIG_INLINE_READ_UNLOCK_IRQ=y
CONFIG_INLINE_READ_UNLOCK_IRQRESTORE=y
CONFIG_INLINE_WRITE_TRYLOCK=y
CONFIG_INLINE_WRITE_LOCK=y
CONFIG_INLINE_WRITE_LOCK_BH=y
CONFIG_INLINE_WRITE_LOCK_IRQ=y
CONFIG_INLINE_WRITE_LOCK_IRQSAVE=y
CONFIG_INLINE_WRITE_UNLOCK=y
CONFIG_INLINE_WRITE_UNLOCK_BH=y
CONFIG_INLINE_WRITE_UNLOCK_IRQ=y
CONFIG_INLINE_WRITE_UNLOCK_IRQRESTORE=y
CONFIG_MUTEX_SPIN_ON_OWNER=y
CONFIG_FREEZER=y
# System Type
CONFIG_MMU=y
# CONFIG_ARCH_AAEC2000 is not set
# CONFIG_ARCH_INTEGRATOR is not set
# CONFIG_ARCH_REALVIEW is not set
# CONFIG_ARCH_VERSATILE is not set
# CONFIG_ARCH_AT91 is not set
# CONFIG_ARCH_BCMRING is not set
# CONFIG_ARCH_CLPS711X is not set
# CONFIG_ARCH_CNS3XXX is not set
# CONFIG_ARCH_GEMINI is not set
# CONFIG_ARCH_PRIMA2 is not set
# CONFIG_ARCH_EBSA110 is not set
# CONFIG_ARCH_EP93XX is not set
# CONFIG_ARCH_FOOTBRIDGE is not set
# CONFIG_ARCH_MXC is not set
# CONFIG_ARCH_MXS is not set
# CONFIG_ARCH_NETX is not set
# CONFIG_ARCH_H720X is not set
# CONFIG_ARCH_IOP13XX is not set
# CONFIG_ARCH_IOP32X is not set
# CONFIG_ARCH_IOP33X is not set
# CONFIG_ARCH_IXP23XX is not set
# CONFIG_ARCH_IXP2000 is not set
# CONFIG_ARCH_IXP4XX is not set
# CONFIG_ARCH_L7200 is not set
# CONFIG_ARCH_DOVE is not set
# CONFIG_ARCH_KIRKWOOD is not set
# CONFIG_ARCH_LPC32XX is not set
# CONFIG_ARCH_MV78XX0 is not set
# CONFIG_ARCH_ORION5X is not set
# CONFIG_ARCH_MMP is not set
# CONFIG_ARCH_KS8695 is not set
# CONFIG_ARCH_W90X900 is not set
# CONFIG_ARCH_TEGRA is not set
# CONFIG_ARCH_PICOXCELL is not set
# CONFIG_ARCH_PNX4008 is not set
# CONFIG_ARCH_PXA is not set
# CONFIG_ARCH_MSM is not set
# CONFIG_ARCH_SHMOBILE is not set
# CONFIG_ARCH_RPC is not set
# CONFIG_ARCH_SA1100 is not set
# CONFIG_ARCH_S3C2410 is not set
# CONFIG_ARCH_S3C64XX is not set
# CONFIG_ARCH_S5P64X0 is not set
# CONFIG_ARCH_S5PC100 is not set
# CONFIG_ARCH_S5PV210 is not set
# CONFIG_ARCH_EXYNOS4 is not set
# CONFIG_ARCH_SHARK is not set
# CONFIG_ARCH_TCC_926 is not set
# CONFIG_ARCH_U300 is not set
# CONFIG_ARCH_U8500 is not set
# CONFIG_ARCH_NOMADIK is not set
# CONFIG_ARCH_DAVINCI is not set
# CONFIG_ARCH_OMAP is not set
# CONFIG_PLAT_SPEAR is not set
# CONFIG_ARCH_VT8500 is not set
# CONFIG_ARCH_ZYNQ is not set
# CONFIG_GPIO_PCA953X is not set
# CONFIG_KEYBOARD_GPIO_POLLED is not set
# CONFIG_KEYBOARD_TCA6416 is not set
# CONFIG_KEYBOARD_MATRIX is not set
# CONFIG_KEYBOARD_LM8323 is not set
# CONFIG_KEYBOARD_MAX7359 is not set
# CONFIG_KEYBOARD_MCS is not set
# CONFIG_KEYBOARD_MPR121 is not set
# CONFIG_KEYBOARD_NEWTON is not set
# CONFIG_KEYBOARD_OPENCORES is not set
# CONFIG_KEYBOARD_STOWAWAY is not set
# CONFIG_KEYBOARD_SUNKBD is not set
# CONFIG_KEYBOARD_XTKBD is not set
# CONFIG_INPUT_MOUSE is not set
# CONFIG_INPUT_JOYSTICK is not set
# CONFIG_INPUT_TABLET is not set
# CONFIG_INPUT_TOUCHSCREEN is not set
# CONFIG_INPUT_MISC is not set
# CONFIG_SERIO_I8042 is not set
# CONFIG_SERIO_SERPORT is not set
# CONFIG_SERIO_PCIPS2 is not set
# CONFIG_SERIO_LIBPS2 is not set
# CONFIG_SERIO_RAW is not set
# CONFIG_SERIO_ALTERA_PS2 is not set
# CONFIG_SERIO_PS2MULT is not set
# CONFIG_GAMEPORT is not set
# Character devices
CONFIG_VT=y
CONFIG_CONSOLE_TRANSLATIONS=y
CONFIG_VT_CONSOLE=y
CONFIG_HW_CONSOLE=y
CONFIG_VT_HW_CONSOLE_BINDING=y
CONFIG_DEVKMEM=y
# Serial drivers
CONFIG_SERIAL_8250=y
CONFIG_SERIAL_8250_CONSOLE=y
CONFIG_SERIAL_8250_PCI=y
CONFIG_SERIAL_8250_NR_UARTS=4
CONFIG_SERIAL_8250_RUNTIME_UARTS=4
CONFIG_SERIAL_8250_EXTENDED=y
CONFIG_SERIAL_8250_MANY_PORTS=y
CONFIG_SERIAL_8250_SHARE_IRQ=y
CONFIG_SERIAL_8250_DETECT_IRQ=y
CONFIG_SERIAL_8250_RSA=y
# Non-8250 serial port support
CONFIG_SERIAL_CORE=y
CONFIG_SERIAL_CORE_CONSOLE=y
CONFIG_SERIAL_JSM=y
CONFIG_UNIX98_PTYS=y
# CONFIG_DEVPTS_MULTIPLE_INSTANCES is not set
CONFIG_LEGACY_PTYS=y
CONFIG_LEGACY_PTY_COUNT=256
CONFIG_PRINTER=y
CONFIG_LP_CONSOLE=y
CONFIG_PPDEV=y
CONFIG_HVC_DRIVER=y
CONFIG_VIRTIO_CONSOLE=y
CONFIG_IPMI_HANDLER=y
CONFIG_IPMI_PANIC_EVENT=y
CONFIG_IPMI_DEVICE_INTERFACE=y
CONFIG_IPMI_SI=y
CONFIG_IPMI_WATCHDOG=y
CONFIG_IPMI_POWEROFF=y
CONFIG_HW_RANDOM=y
CONFIG_HW_RANDOM_TIMERIOMEM=y
CONFIG_HW_RANDOM_INTEL=y
CONFIG_HW_RANDOM_AMD=y
CONFIG_HW_RANDOM_VIA=y
CONFIG_HW_RANDOM_VIRTIO=y
CONFIG_NVRAM=y
CONFIG_R3964=y
CONFIG_APPLICOM=y
CONFIG_MWAVE=y
CONFIG_RAW_DRIVER=y
CONFIG_MAX_RAW_DEVS=256
CONFIG_HPET=y
# CONFIG_HPET_MMAP is not set
CONFIG_HANGCHECK_TIMER=y
CONFIG_TCG_TPM=y
CONFIG_TCG_TIS=y
CONFIG_TCG_NSC=y
CONFIG_TCG_ATMEL=y
CONFIG_TCG_INFINEON=y
CONFIG_TCG_XEN=y
CONFIG_TELCLOCK=y
CONFIG_DEVPORT=y
CONFIG_I2C=y
CONFIG_I2C_BOARDINFO=y
CONFIG_I2C_COMPAT=y
CONFIG_I2C_CHARDEV=y
CONFIG_I2C_HELPER_AUTO=y
CONFIG_I2C_ALGOBIT=y
# I2C Hardware Bus support
# CONFIG_I2C_ALI1535 is not set
# CONFIG_I2C_ALI1563 is not set
# CONFIG_I2C_ALI15X3 is not set
# CONFIG_I2C_AMD756 is not set
# CONFIG_I2C_AMD8111 is not set
# CONFIG_I2C_I801 is not set
# CONFIG_I2C_ISCH is not set
# CONFIG_I2C_PIIX4 is not set
# CONFIG_I2C_NFORCE2 is not set
# CONFIG_I2C_SIS5595 is not set
# CONFIG_I2C_SIS630 is not set
# CONFIG_I2C_SIS96X is not set
# CONFIG_I2C_VIA is not set
# CONFIG_I2C_VIAPRO is not set
# CONFIG_I2C_OCORES is not set
# CONFIG_I2C_PARPORT_LIGHT is not set
# CONFIG_I2C_TAOS_EVM is not set
# CONFIG_I2C_TINY_USB is not set
# CONFIG_I2C_PCA_PLATFORM is not set
# CONFIG_I2C_STUB is not set
# CONFIG_I2C_DEBUG_CORE is not set
# CONFIG_I2C_DEBUG_ALGO is not set
# CONFIG_I2C_DEBUG_BUS is not set
# CONFIG_SPI is not set
# CONFIG_HSI is not set
# PPS support
CONFIG_PPS=y
# CONFIG_PPS_DEBUG is not set
# CONFIG_NTP_PPS is not set
# PPS generators support
# CONFIG_PPS_GENERATOR_PARPORT is not set
# CONFIG_W1 is not set
CONFIG_POWER_SUPPLY=y
# CONFIG_POWER_SUPPLY_DEBUG is not set
# CONFIG_PDA_POWER is not set
# CONFIG_BATTERY_DS2760 is not set
# CONFIG_BATTERY_DS2782 is not set
# CONFIG_BATTERY_BQ20Z75 is not set
# CONFIG_BATTERY_BQ27x00 is not set
# CONFIG_BATTERY_MAX17040 is not set
# CONFIG_BATTERY_MAX17042 is not set
# CONFIG_CHARGER_ISP1704 is not set
# CONFIG_CHARGER_MAX8903 is not set
# CONFIG_CHARGER_GPIO is not set
CONFIG_HWMON=y
# CONFIG_HWMON_VID is not set
# CONFIG_HWMON_DEBUG_CHIP is not set
# Native drivers
# CONFIG_SENSORS_ABITUGURU is not set
# CONFIG_SENSORS_ABITUGURU3 is not set
# CONFIG_SENSORS_AD7414 is not set
# CONFIG_SENSORS_AD7418 is not set
# CONFIG_SENSORS_ADM1021 is not set
# CONFIG_SENSORS_ADM1025 is not set
# CONFIG_SENSORS_ADM1026 is not set
# CONFIG_SENSORS_ADM1029 is not set
# CONFIG_SENSORS_ADM1031 is not set
# CONFIG_SENSORS_ADM9240 is not set
# CONFIG_SENSORS_ADT7411 is not set
# CONFIG_SENSORS_ADT7462 is not set
# CONFIG_SENSORS_ADT7470 is not set
# CONFIG_SENSORS_ADT7475 is not set
# CONFIG_SENSORS_ASC7621 is not set
# CONFIG_SENSORS_K8TEMP is not set
# CONFIG_SENSORS_K10TEMP is not set
# CONFIG_SENSORS_FAM15H_POWER is not set
# CONFIG_SENSORS_ASB100 is not set
# CONFIG_SENSORS_ATXP1 is not set
# CONFIG_SENSORS_DS620 is not set
# CONFIG_SENSORS_DS1621 is not set
# CONFIG_SENSORS_I5K_AMB is not set
# CONFIG_SENSORS_F71805F is not set
# CONFIG_SENSORS_F71882FG is not set
# CONFIG_SENSORS_F75375S is not set
# CONFIG_SENSORS_FSCHMD is not set
# CONFIG_SENSORS_G760A is not set
# CONFIG_SENSORS_GL518SM is not set
# CONFIG_SENSORS_GL520SM is not set
# CONFIG_SENSORS_IBMAEM is not set
# CONFIG_SENSORS_IBMPEX is not set
# CONFIG_SENSORS_IT87 is not set
# CONFIG_SENSORS_JC42 is not set
# CONFIG_SENSORS_LINEAGE is not set
# CONFIG_SENSORS_LM63 is not set
# CONFIG_SENSORS_LM70 is not set
# CONFIG_SENSORS_LM73 is not set
# CONFIG_SENSORS_LM75 is not set
# CONFIG_SENSORS_LM77 is not set
# CONFIG_SENSORS_LM78 is not set
# CONFIG_SENSORS_LM80 is not set
# CONFIG_SENSORS_LM83 is not set
# CONFIG_SENSORS_LM85 is not set
# CONFIG_SENSORS_LM87 is not set
# CONFIG_SENSORS_LM90 is not set
# CONFIG_SENSORS_LM92 is not set
# CONFIG_SENSORS_LM93 is not set
# CONFIG_SENSORS_LTC4151 is not set
# CONFIG_SENSORS_LTC4215 is not set
# CONFIG_SENSORS_LTC4245 is not set
# CONFIG_SENSORS_LTC4261 is not set
# CONFIG_SENSORS_MAX1111 is not set
# CONFIG_SENSORS_MAX16065 is not set
# CONFIG_SENSORS_MAX1619 is not set
# CONFIG_SENSORS_MAX1668 is not set
# CONFIG_SENSORS_MAX6639 is not set
# CONFIG_SENSORS_MAX6642 is not set
# CONFIG_SENSORS_MAX6650 is not set
# CONFIG_SENSORS_MAX6697 is not set
# CONFIG_SENSORS_MCP3021 is not set
# CONFIG_SENSORS_NCT6775 is not set
# CONFIG_SENSORS_PC87360 is not set
# CONFIG_SENSORS_PC87427 is not set
# CONFIG_SENSORS_PCF8591 is not set
# CONFIG_SENSORS_SHT15 is not set
# CONFIG_SENSORS_SHT21 is not set
# CONFIG_SENSORS_SIS5595 is not set
# CONFIG_SENSORS_SMM665 is not set
# CONFIG_SENSORS_DME1737 is not set
# CONFIG_SENSORS_EMC1403 is not set
# CONFIG_SENSORS_EMC2103 is not set
# CONFIG_SENSORS_SMSC47M1 is not set
# CONFIG_SENSORS_SMSC47M192 is not set
# CONFIG_SENSORS_SMSC47B397 is not set
# CONFIG_SENSORS_SCH56XX_COMMON is not set
# CONFIG_SENSORS_SCH5627 is not set
# CONFIG_SENSORS_SCH5636 is not set
# CONFIG_SENSORS_ADS1015 is not set
# CONFIG_SENSORS_ADS7828 is not set
# CONFIG_SENSORS_AMC6821 is not set
# CONFIG_SENSORS_THMC50 is not set
# CONFIG_SENSORS_TMP102 is not set
# CONFIG_SENSORS_TMP401 is not set
# CONFIG_SENSORS_TMP421 is not set
# CONFIG_SENSORS_VIA_CPUTEMP is not set
# CONFIG_SENSORS_VIA686A is not set
# CONFIG_SENSORS_VT1211 is not set
# CONFIG_SENSORS_VT8231 is not set
# CONFIG_SENSORS_W83781D is not set
# CONFIG_SENSORS_W83791D is not set
# CONFIG_SENSORS_W83792D is not set
# CONFIG_SENSORS_W83793 is not set
# CONFIG_SENSORS_W83795 is not set
# CONFIG_SENSORS_W83L785TS is not set
# CONFIG_SENSORS_W83L786NG is not set
# CONFIG_SENSORS_W83627HF is not set
# CONFIG_SENSORS_W83627EHF is not set
# CONFIG_SENSORS_ULTRA45 is not set
# CONFIG_SENSORS_HDAPS is not set
# CONFIG_SENSORS_APPLESMC is not set
# CONFIG_THERMAL is not set
# CONFIG_WATCHDOG is not set
CONFIG_SSB_POSSIBLE=y
# Sonics Silicon Backplane
# CONFIG_SSB is not set
CONFIG_BCMA_POSSIBLE=y
# Broadcom specific AMBA
# CONFIG_BCMA is not set
# Multifunction device drivers
# CONFIG_MFD_CORE is not set
# CONFIG_MFD_88PM860X is not set
# CONFIG_MFD_SM501 is not set
# CONFIG_MFD_ASIC3 is not set
# CONFIG_HTC_EGPIO is not set
# CONFIG_HTC_PASIC3 is not set
# CONFIG_HTC_I2CPLD is not set
# CONFIG_TPS6105X is not set
# CONFIG_TPS65010 is not set
# CONFIG_TPS6507X is not set
# CONFIG_MFD_TPS65217 is not set
# CONFIG_MFD_TPS6586X is not set
# CONFIG_MFD_TPS65910 is not set
# CONFIG_MFD_TPS65912_I2C is not set
# CONFIG_MFD_TPS65912_SPI is not set
# CONFIG_TWL4030_CORE is not set
# CONFIG_TWL6040_CORE is not set
# CONFIG_MFD_STMPE is not set
# CONFIG_MFD_TC3589X is not set
# CONFIG_MFD_TMIO is not set
# CONFIG_MFD_T7L66XB is not set
# CONFIG_MFD_TC6387XB is not set
# CONFIG_MFD_TC6393XB is not set
# CONFIG_PMIC_DA903X is not set
# CONFIG_PMIC_ADP5520 is not set
# CONFIG_MFD_MAX8925 is not set
# CONFIG_MFD_MAX8997 is not set
# CONFIG_MFD_MAX8998 is not set
# CONFIG_MFD_S5M_CORE is not set
# CONFIG_MFD_WM8400 is not set
# CONFIG_MFD_WM831X_I2C is not set
# CONFIG_MFD_WM831X_SPI is not set
# CONFIG_MFD_WM8350_I2C is not set
# CONFIG_MFD_WM8994 is not set
# CONFIG_MFD_PCF50633 is not set
# CONFIG_MFD_MC13XXX is not set
# CONFIG_ABX500_CORE is not set
# CONFIG_EZX_PCAP is not set
# CONFIG_MFD_CS5535 is not set
# CONFIG_MFD_TIMBERDALE is not set
# CONFIG_LPC_SCH is not set
# CONFIG_MFD_RDC321X is not set
# CONFIG_MFD_JANZ_CMODIO is not set
# CONFIG_MFD_VX855 is not set
# CONFIG_MFD_WL1273_CORE is not set
# CONFIG_MFD_TPS65090 is not set
# CONFIG_MFD_AAT2870_CORE is not set
# CONFIG_MFD_RC5T583 is not set
# CONFIG_MFD_VIPERBOARD is not set
# CONFIG_REGULATOR is not set
# CONFIG_MEDIA_SUPPORT is not set
# Graphics support
# CONFIG_VGA_ARB is not set
# CONFIG_DRM is not set
# CONFIG_VGASTATE is not set
# CONFIG_VIDEO_OUTPUT_CONTROL is not set
# CONFIG_FB is not set
# CONFIG_BACKLIGHT_LCD_SUPPORT is not set
# Display device support
# CONFIG_DISPLAY_SUPPORT is not set
# Console display driver support
CONFIG_DUMMY_CONSOLE=y
CONFIG_FRAMEBUFFER_CONSOLE=y
CONFIG_FRAMEBUFFER_CONSOLE_DETECT_PRIMARY=y
CONFIG_FRAMEBUFFER_CONSOLE_ROTATION=y
# CONFIG_FONTS is not set
CONFIG_FONT_8x8=y
CONFIG_FONT_8x16=y
# CONFIG_LOGO is not set
# CONFIG_SOUND is not set
CONFIG_HID_SUPPORT=y
CONFIG_HID=y
# CONFIG_HID_BATTERY_STRENGTH is not set
# CONFIG_HIDRAW is not set
# CONFIG_HID_PID is not set
CONFIG_USB_HID=y
# CONFIG_USB_HIDDEV is not set
# USB HID Boot Protocol drivers
# CONFIG_USB_KBD is not set
# CONFIG_USB_MOUSE is not set
# Special HID drivers
CONFIG_HID_A4TECH=y
CONFIG_HID_ACRUX=y
CONFIG_HID_ACRUX_FF=y
CONFIG_HID_APPLE=y
CONFIG_HID_BELKIN=y
CONFIG_HID_CHERRY=y
CONFIG_HID_CHICONY=y
CONFIG_HID_PRODIKEYS=y
CONFIG_HID_CYPRESS=y
CONFIG_HID_DRAGONRISE=y
CONFIG_DRAGONRISE_FF=y
CONFIG_HID_EMS_FF=y
CONFIG_HID_ELECOM=y
CONFIG_HID_EZKEY=y
CONFIG_HID_KEYTOUCH=y
CONFIG_HID_KYE=y
CONFIG_HID_UCLOGIC=y
CONFIG_HID_WALTOP=y
CONFIG_HID_GYRATION=y
CONFIG_HID_TWINHAN=y
CONFIG_HID_KENSINGTON=y
CONFIG_HID_LCPOWER=y
CONFIG_HID_LOGITECH=y
CONFIG_LOGITECH_FF=y
CONFIG_LOGIRUMBLEPAD2_FF=y
CONFIG_LOGIG940_FF=y
CONFIG_LOGIWII_FF=y
CONFIG_HID_MAGICMOUSE=y
CONFIG_HID_MICROSOFT=y
CONFIG_HID_MONTEREY=y
CONFIG_HID_MULTITOUCH=y
CONFIG_HID_NTRIG=y
CONFIG_HID_ORTEK=y
CONFIG_HID_PANTHERLORD=y
CONFIG_PANTHERLORD_FF=y
CONFIG_HID_PETALYNX=y
CONFIG_HID_PICOLCD=y
CONFIG_HID_PICOLCD_FB=y
CONFIG_HID_PICOLCD_BACKLIGHT=y
CONFIG_HID_PICOLCD_LCD=y
CONFIG_HID_PICOLCD_LEDS=y
CONFIG_HID_PICOLCD_CIR=y
CONFIG_HID_PLANTRONICS=y
CONFIG_HID_PRIMAX=y
CONFIG_HID_ROCCAT=y
CONFIG_HID_SAITEK=y
CONFIG_HID_SAMSUNG=y
CONFIG_HID_SONY=y
CONFIG_HID_SPEEDLINK=y
CONFIG_HID_SUNPLUS=y
CONFIG_HID_GREENASIA=y
CONFIG_GREENASIA_FF=y
CONFIG_HID_SMARTJOYPLUS=y
CONFIG_SMARTJOYPLUS_FF=y
CONFIG_HID_TIVO=y
CONFIG_HID_TOPSEED=y
CONFIG_HID_THRUSTMASTER=y
CONFIG_HID_WACOM=y
CONFIG_HID_WIIMOTE=y
CONFIG_HID_XINMO=y
CONFIG_HID_ZEROPLUS=y
CONFIG_ZEROPLUS_FF=y
CONFIG_HID_ZYDACRON=y
CONFIG_USB_SUPPORT=y
CONFIG_USB_COMMON=y
CONFIG_USB_ARCH_HAS_HCD=y
CONFIG_USB_ARCH_HAS_OHCI=y
CONFIG_USB_ARCH_HAS_EHCI=y
CONFIG_USB=y
# CONFIG_USB_DEBUG is not set
# CONFIG_USB_ANNOUNCE_NEW_DEVICES is not set
# Miscellaneous USB options
CONFIG_USB_DEVICEFS=y
CONFIG_USB_DEVICE_CLASS=y
# CONFIG_USB_DYNAMIC_MINORS is not set
CONFIG_USB_SUSPEND=y
# CONFIG_USB_OTG is not set
# CONFIG_USB_MON is not set
# CONFIG_USB_WUSB is not set
# CONFIG_USB_WUSB_CBAF is not set
# USB Host Controller Drivers
# CONFIG_USB_C67X00_HCD is not set
CONFIG_USB_EHCI_HCD=y
CONFIG_USB_EHCI_ROOT_HUB_TT=y
CONFIG_USB_EHCI_TT_NEWSCHED=y
# CONFIG_USB_OXU210HP_HCD is not set
# CONFIG_USB_ISP116X_HCD is not set
# CONFIG_USB_ISP1760_HCD is not set
# CONFIG_USB_ISP1362_HCD is not set
# CONFIG_USB_OHCI_HCD is not set
# CONFIG_USB_SL811_HCD is not set
# CONFIG_USB_R8A66597_HCD is not set
# CONFIG_USB_WHCI_HCD is not set
# CONFIG_USB_HWA_HCD is not set
# USB Device Class drivers
# CONFIG_USB_ACM is not set
# CONFIG_USB_PRINTER is not set
# CONFIG_USB_WDM is not set
# CONFIG_USB_TMC is not set
# NOTE: USB_STORAGE depends on SCSI but BLK_DEV_SD may
# NOTE: be set to m while USB_STORAGE is set to y, this can
# NOTE: be avoided by making USB_STORAGE depend on SCSI
# NOTE: instead of SCSI || !SCSI
# CONFIG_USB_STORAGE is not set
# USB Imaging devices
# CONFIG_USB_MDC800 is not set
# CONFIG_USB_MICROTEK is not set
# CONFIG_USB_MON is not set
# CONFIG_USB_MUSB_HDRC is not set
# USB Physical Layer drivers
# CONFIG_USB_ISP1301 is not set
# CONFIG_USB_GADGET is not set
# CONFIG_UWB is not set
# CONFIG_MMC is not set
# CONFIG_MEMSTICK is not set
# CONFIG_NEW_LEDS is not set
# CONFIG_ACCESSIBILITY is not set
# CONFIG_INFINIBAND is not set
# CONFIG_EDAC is not set
CONFIG_RTC_LIB=y
CONFIG_RTC_CLASS=y
CONFIG_RTC_HCTOSYS=y
CONFIG_RTC_HCTOSYS_DEVICE="rtc0"
# CONFIG_RTC_DEBUG is not set
# RTC interfaces
CONFIG_RTC_INTF_SYSFS=y
CONFIG_RTC_INTF_PROC=y
CONFIG_RTC_INTF_DEV=y
# CONFIG_RTC_INTF_DEV_UIE_EMUL is not set
# CONFIG_RTC_DRV_TEST is not set
# SPI RTC drivers
# I2C RTC drivers
# CONFIG_RTC_DRV_DS1307 is not set
# CONFIG_RTC_DRV_DS1374 is not set
# CONFIG_RTC_DRV_DS1672 is not set
# CONFIG_RTC_DRV_DS3232 is not set
# CONFIG_RTC_DRV_MAX6900 is not set
# CONFIG_RTC_DRV_RS5C372 is not set
# CONFIG_RTC_DRV_ISL1208 is not set
# CONFIG_RTC_DRV_ISL12022 is not set
# CONFIG_RTC_DRV_X1205 is not set
# CONFIG_RTC_DRV_PCF8563 is not set
# CONFIG_RTC_DRV_PCF8583 is not set
# CONFIG_RTC_DRV_M41T80 is not set
# CONFIG_RTC_DRV_BQ32K is not set
# CONFIG_RTC_DRV_S35390A is not set
# CONFIG_RTC_DRV_FM3130 is not set
# CONFIG_RTC_DRV_RX8581 is not set
# CONFIG_RTC_DRV_RX8025 is not set
# CONFIG_RTC_DRV_EM3027 is not set
# CONFIG_RTC_DRV_RV3029C2 is not set
# SPI RTC drivers
# Platform RTC drivers
# CONFIG_RTC_DRV_CMOS is not set
# CONFIG_RTC_DRV_DS1286 is not set
# CONFIG_RTC_DRV_DS1511 is not set
# CONFIG_RTC_DRV_DS1553 is not set
# CONFIG_RTC_DRV_DS1742 is not set
# CONFIG_RTC_DRV_STK17TA8 is not
```

```
                                                
```

```
# Auto-generated config file
# Usage of ./config:
  -alsologtostderr
    	log to standard error as well as files
  -config string
    	Configuration file to load
  -dump-config
    	Dump current configuration and exit
  -help
    	Show help
  -log_backtrace_at value
    	when logging hits line file:N, emit a stack trace
  -log_dir string
    	If non-empty, write log files in this directory
  -logtostderr
    	log to standard error instead of files
  -port int
    	Port to listen on (default 8080)
  -stderrthreshold value
    	logs at or above this threshold go to stderr
  -v value
    	log level for V logs
  -vmodule value
    	comma-separated list of pattern=N settings for file-filtered logging
# Default configuration
server:
  host: "0.0.0.0"
  port: 8080
  read_timeout: 30s
  write_timeout: 30s
  idle_timeout: 120s
  max_header_bytes: 1048576
  tls:
    enabled: false
    cert_file: ""
    key_file: ""
    min_version: "1.2"
database:
  driver: postgres
  host: localhost
  port: 5432
  name: myapp
  user: myapp
  password: ""
  sslmode: disable
  max_open_conns: 25
  max_idle_conns: 5
  conn_max_lifetime: 5m
  migrations_dir: ./migrations
redis:
  addr: localhost:6379
  password: ""
  db: 0
  pool_size: 10
  dial_timeout: 5s
  read_timeout: 3s
  write_timeout: 3s
logging:
  level: info
  format: text
  output: stdout
  file_path: ""
  max_size_mb: 100
  max_backups: 3
  max_age_days: 28
  compress: true
auth:
  jwt_secret: ""
  jwt_issuer: myapp
  access_token_ttl: 15m
  refresh_token_ttl: 720h
  bcrypt_cost: 12
  session_cookie_name: session
  session_cookie_secure: true
  session_cookie_http_only: true
  session_cookie_same_site: lax
cors:
  allowed_origins:
    - "*"
  allowed_methods:
    - GET
    - POST
    - PUT
    - PATCH
    - DELETE
    - OPTIONS
  allowed_headers:
    - Content-Type
    - Authorization
    - X-Request-ID
  exposed_headers: []
  allow_credentials: false
  max_age: 300
rate_limit:
  enabled: true
  requests_per_second: 10
  burst: 20
  by: ip
metrics:
  enabled: true
  path: /metrics
  namespace: myapp
tracing:
  enabled: false
  exporter: otlp
  endpoint: localhost:4317
  sample_ratio: 0.1
  service_name: myapp
feature_flags:
  new_dashboard: false
  beta_api: false
  experimental_search: false
# Auto-generated config file
# Usage of ./config:
  -alsologtostderr
    	log to standard error as well as files
  -config string
    	Configuration file to load
  -dump-config
    	Dump current configuration and exit
  -help
    	Show help
  -log_backtrace_at value
    	when logging hits line file:N, emit a stack trace
  -log_dir string
    	If non-empty, write log files in this directory
  -logtostderr
    	log to standard error instead of files
  -port int
    	Port to listen on (default 8080)
  -stderrthreshold value
    	logs at or above this threshold go to stderr
  -v value
    	log level for V logs
  -vmodule value
    	comma-separated list of pattern=N settings for file-filtered logging
# Default configuration
server:
  host: "0.0.0.0"
  port: 8080
  read_timeout: 30s
  write_timeout: 30s
  idle_timeout: 120s
  max_header_bytes: 1048576
  tls:
    enabled: false
    cert_file: ""
    key_file: ""
    min_version: "1.2"
database:
  driver: postgres
  host: localhost
  port: 5432
  name: myapp
  user: myapp
  password: ""
  sslmode: disable
  max_open_conns: 25
  max_idle_conns: 5
  conn_max_lifetime: 5m
  migrations_dir: ./migrations
redis:
  addr: localhost:6379
  password: ""
  db: 0
  pool_size: 10
  dial_timeout: 5s
  read_timeout: 3s
  write_timeout: 3s
logging:
  level: info
  format: text
  output: stdout
  file_path: ""
  max_size_mb: 100
  max_backups: 3
  max_age_days: 28
  compress: true
auth:
  jwt_secret: ""
  jwt_issuer: myapp
  access_token_ttl: 15m
  refresh_token_ttl: 720h
  bcrypt_cost: 12
  session_cookie_name: session
  session_cookie_secure: true
  session_cookie_http_only: true
  session_cookie_same_site: lax
cors:
  allowed_origins:
    - "*"
  allowed_methods:
    - GET
    - POST
    - PUT
    - PATCH
    - DELETE
    - OPTIONS
  allowed_headers:
    - Content-Type
    - Authorization
    - X-Request-ID
  exposed_headers: []
  allow_credentials: false
  max_age: 300
rate_limit:
  enabled: true
  requests_per_second: 10
  burst: 20
  by: ip
metrics:
  enabled: true
  path: /metrics
  namespace: myapp
tracing:
  enabled: false
  exporter: otlp
  endpoint: localhost:4317
  sample_ratio: 0.1
  service_name: myapp
feature_flags:
  new_dashboard: false
  beta_api: false
  experimental_search: false
```

```
```

```
```

```
```

```
# Configuration
# ------------
#
# Variables with empty strings are commented out
#
# Options set explicitly to the compiled in default are commented out
#
# Lines beginning with # are comments
#
# Options for the network layer
#
# Address to listen on (ipv4, ipv6 or hostname)
# Multiple addresses may be specified, for example:
# ListenAddress 0.0.0.0 ::
#
# ListenAddress [hostname]
#
# Port to listen on
# Port 22
#
# The default is to listen on both IPv4 and IPv6, where available
# AddressFamily any
#
# Limit the number of connections from the same IP address
# for the purpose of avoiding DoS attacks
# PerSourceMaxStartups 10
#
# Restrict the number of connections to the server
# MaxStartups 10:30:100
#
# Enable pam support
# UsePAM yes
#
# Enable forwarding of connections to the PAM subsystem to
# circumvent restrictions on setuid binaries
# PAMServiceName sshd
#
# Enable forwarding of the authentication agent connection
# AllowAgentForwarding yes
#
# Enable forwarding of arbitrary TCP connections
# AllowTcpForwarding yes
#
# Allow remote hosts to connect to local forwarded ports
# GatewayPorts no
#
# Specify the maximum number of authentication attempts per connection
# MaxAuthTries 6
#
# Specify the maximum number of sessions per network connection
# MaxSessions 10
#
# Control the order in which hostkeys are preferred
# HostKeyAlgorithms ssh-ed25519,ssh-ed25519-cert-v01@openssh.com,sk-ssh-ed25519@openssh.com,sk-ssh-ed25519-cert-v01@openssh.com,rsa-sha2-512,rsa-sha2-512-cert-v01@openssh.com,rsa-sha2-256,rsa-sha2-256-cert-v01@openssh.com
#
# Specify the key exchange algorithms
# KexAlgorithms curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521,diffie-hellman-group-exchange-sha256,diffie-hellman-group16-sha512,diffie-hellman-group18-sha512,diffie-hellman-group14-sha256,diffie-hellman-group14-sha1
#
# Specify the ciphers allowed
# Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr
#
# Specify the maximum amount of data that may be transmitted before the session key is renegotiated
# RekeyLimit 1G 1h
#
# Specify how many seconds the server will wait for a login to complete
# LoginGraceTime 120
#
# Limit access to specific interfaces or addresses
# ListenAddress 0.0.0.0
#
# Specify the maximum number of authentication attempts per connection
# MaxAuthTries 6
#
# Specify the maximum number of sessions per network connection
# MaxSessions 10
```

```
type PaymentProcessor struct {
    client     *http.Client
    apiKey     string
    baseURL    string
    maxRetries int
    timeout    time.Duration
}

func NewPaymentProcessor(apiKey string) *PaymentProcessor {
    return &PaymentProcessor{
        client:     &http.Client{Timeout: 30 * time.Second},
        apiKey:     apiKey,
        baseURL:    "https://api.payments.example.com/v2",
        maxRetries: 3,
        timeout:    30 * time.Second,
    }
}

func (p *PaymentProcessor) ProcessPayment(ctx context.Context, req PaymentRequest) (*PaymentResponse, error) {
    if req.Amount <= 0 {
        return nil, fmt.Errorf("invalid amount: %d", req.Amount)
    }
    if req.Currency == "" {
        return nil, fmt.Errorf("currency is required")
    }

    body, err := json.Marshal(req)
    if err != nil {
        return nil, fmt.Errorf("marshal request: %w", err)
    }

    var lastErr error
    for attempt := 0; attempt <= p.maxRetries; attempt++ {
        if attempt > 0 {
            backoff := time.Duration(1<<uint(attempt-1)) * time.Second
            select {
            case <-ctx.Done():
                return nil, ctx.Err()
            case <-time.After(backoff):
            }
        }

        httpReq, err := http.NewRequestWithContext(ctx, "POST", p.baseURL+"/charges", bytes.NewReader(body))
        if err != nil {
            return nil, fmt.Errorf("create request: %w", err)
        }
        httpReq.Header.Set("Authorization", "Bearer "+p.apiKey)
        httpReq.Header.Set("Content-Type", "application/json")
        httpReq.Header.Set("Idempotency-Key", req.IdempotencyKey)

        resp, err := p.client.Do(httpReq)
        if err != nil {
            lastErr = err
            continue
        }
        defer resp.Body.Close()

        if resp.StatusCode >= 500 {
            lastErr = fmt.Errorf("server error: %d", resp.StatusCode)
            continue
        }

        var result PaymentResponse
        if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
            return nil, fmt.Errorf("decode response: %w", err)
        }
        return &result, nil
    }
    return nil, fmt.Errorf("max retries exceeded: %w", lastErr)
}

type PaymentRequest struct {
    Amount         int64  `json:"amount"`
    Currency       string `json:"currency"`
    IdempotencyKey string `json:"idempotency_key"`
}

type PaymentResponse struct {
    ID     string `json:"id"`
    Status string `json:"status"`
}
```

```
#!/usr/bin/env bash
set -euo pipefail

readonly SCRIPT_NAME="$(basename "$0")"
readonly LOG_FILE="/var/log/${SCRIPT_NAME%.sh}.log"

log() {
    local level="$1"; shift
    printf '%s [%s] %s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$level" "$*" | tee -a "$LOG_FILE"
}

die() {
    log ERROR "$*"
    exit 1
}

usage() {
    cat <<EOF
Usage: $SCRIPT_NAME [-n] [-v] <source> <destination>
  -n  dry run
  -v  verbose
EOF
}

DRY_RUN=0
VERBOSE=0
while getopts ":nvh" opt; do
    case "$opt" in
        n) DRY_RUN=1 ;;
        v) VERBOSE=1 ;;
        h) usage; exit 0 ;;
        *) usage; exit 2 ;;
    esac
done
shift $((OPTIND - 1))

[[ $# -eq 2 ]] || { usage; exit 2; }
SRC="$1"
DST="$2"

[[ -d "$SRC" ]] || die "source is not a directory: $SRC"

run() {
    if (( DRY_RUN )); then
        log DRY "$*"
    else
        (( VERBOSE )) && log RUN "$*"
        "$@"
    fi
}

run rsync -a --delete "$SRC/" "$DST/"
log INFO "sync complete"
```

```
import { useEffect, useRef, useState } from "react";

export function useDebouncedValue<T>(value: T, delayMs: number): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);
  return debounced;
}

export function usePrevious<T>(value: T): T | undefined {
  const ref = useRef<T | undefined>(undefined);
  useEffect(() => {
    ref.current = value;
  }, [value]);
  return ref.current;
}

export function useOnClickOutside(
  ref: React.RefObject<HTMLElement>,
  handler: (event: MouseEvent | TouchEvent) => void,
) {
  useEffect(() => {
    const listener = (event: MouseEvent | TouchEvent) => {
      if (!ref.current || ref.current.contains(event.target as Node)) {
        return;
      }
      handler(event);
    };
    document.addEventListener("mousedown", listener);
    document.addEventListener("touchstart", listener);
    return () => {
      document.removeEventListener("mousedown", listener);
      document.removeEventListener("touchstart", listener);
    };
  }, [ref, handler]);
}

export function useMediaQuery(query: string): boolean {
  const [matches, setMatches] = useState(false);
  useEffect(() => {
    const media = window.matchMedia(query);
    setMatches(media.matches);
    const listener = () => setMatches(media.matches);
    media.addEventListener("change", listener);
    return () => media.removeEventListener("change", listener);
  }, [query]);
  return matches;
}
```

```
import { useEffect, useRef, useState } from "react";

export function useDebouncedValue<T>(value: T, delayMs: number): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);
  return debounced;
}

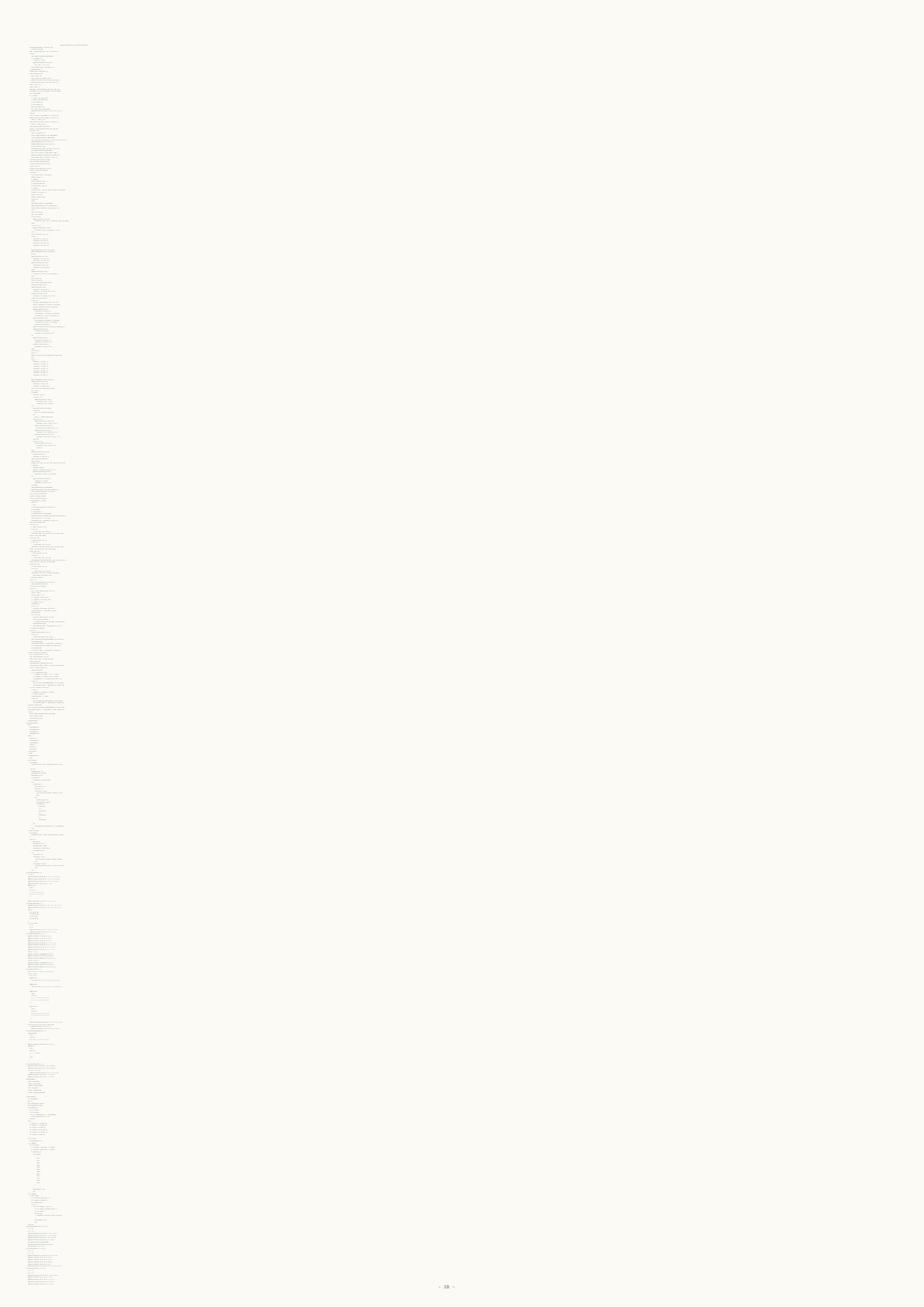
export function usePrevious<T>(value: T): T | undefined {
  const ref = useRef<T | undefined>(undefined);
  useEffect(() => {
    ref.current = value;
  }, [value]);
  return ref.current;
}

export function useOnClickOutside(
  ref: React.RefObject<HTMLElement>,
  handler: (event: MouseEvent | TouchEvent) => void,
) {
  useEffect(() => {
    const listener = (event: MouseEvent | TouchEvent) => {
      if (!ref.current || ref.current.contains(event.target as Node)) {
        return;
      }
      handler(event);
    };
    document.addEventListener("mousedown", listener);
    document.addEventListener("touchstart", listener);
    return () => {
      document.removeEventListener("mousedown", listener);
      document.removeEventListener("touchstart", listener);
    };
  }, [ref, handler]);
}

export function useMediaQuery(query: string): boolean {
  const [matches, setMatches] = useState(false);
  useEffect(() => {
    const media = window.matchMedia(query);
    setMatches(media.matches);
    const listener = () => setMatches(media.matches);
    media.addEventListener("change", listener);
    return () => media.removeEventListener("change", listener);
  }, [query]);
  return matches;
}
```

```
```





```
```

```
# ============================================================
# Development Commands Cheat Sheet — Version 2.4.1 — Build 7291
# ============================================================

# --- Git ---
git status -sb
git log --oneline --graph --decorate -20
git diff --stat HEAD~1
git diff --name-only main...HEAD
git stash push -m "wip: description"
git stash pop
git rebase -i HEAD~5
git cherry-pick <commit-sha>
git bisect start && git bisect bad && git bisect good v2.4.0
git worktree add ../hotfix-branch hotfix/urgent-fix
git clean -fdx -e .env -e .idea/
git reflog --date=iso | head -30
git shortlog -sne --since="2 weeks ago"
git blame -w -C -C -C src/core/engine.c
git log --follow --diff-filter=D --summary | grep delete
git archive --format=tar.gz --prefix=release-2.4.1/ HEAD > release-2.4.1.tar.gz
git submodule update --init --recursive --jobs 4
git config --global alias.lg "log --graph --pretty=format:'%h %s' --abbrev-commit"

# --- Build (CMake) ---
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo -DENABLE_LTO=ON
cmake --build build --parallel $(nproc)
cmake --build build --target install --config Release
cmake --build build --target clean
cmake -LAH build | grep -E "^[A-Z]" | sort
ccache -s
ccache -z
export CC="ccache clang" CXX="ccache clang++"
cmake -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -S . -B build
ln -sf build/compile_commands.json .
cmake --build build --target help | grep -i test
cmake --build build --target package
cpack -G DEB -C Release
cpack -G RPM -C Release
cpack -G TGZ -C Release
cmake --build build --target docs
cmake --build build --target format
cmake --build build --target check-format
cmake --build build --target lint
cmake --build build --target coverage
cmake --build build --target benchmark
cmake --build build --target install/strip
cmake --build build --target uninstall
cmake --build build --target rebuild_cache
cmake --build build --target edit_cache
cmake --build build --target list_install_components
cmake --build build --target package_source
cmake --build build --target test_ARGS="-R unit_ -j 8"
cmake --build build --target test_ARGS="--output-on-failure --timeout 120"
cmake --build build --target test_ARGS="--rerun-failed --output-on-failure"
cmake --build build --target test_ARGS="-L integration"
cmake --build build --target test_ARGS="-E slow_"
cmake --build build --target test_ARGS="--repeat until-fail:3"
cmake --build build --target test_ARGS="--shuffle --seed 42"
cmake --build build --target test_ARGS="--stop-on-failure"
cmake --build build --target test_ARGS="--output-junit results.xml"
cmake --build build --target test_ARGS="--parallel-level 16"
cmake --build build --target test_ARGS="--resource-spec-file resources.json"
cmake --build build --target test_ARGS="--tests-regex '^(foo|bar)_'"
cmake --build build --target test_ARGS="--exclude-regex 'flaky_'"
cmake --build build --target test_ARGS="--timeout 30"
cmake --build build --target test_ARGS="--no-tests=error"
cmake --build build --target test_ARGS="--schedule-random"
cmake --build build --target test_ARGS="--progress"
cmake --build build --target test_ARGS="--quiet"
cmake --build build --target test_ARGS="--verbose"
cmake --build build --target test_ARGS="--debug"
cmake --build build --target test_ARGS="--print-labels"
cmake --build build --target test_ARGS="--label-regex '^(unit|integration)$'"
cmake --build build --target test_ARGS="--label-exclude 'slow|flaky'"
cmake --build build --target test_ARGS="--fixture-exclude-regex 'setup_'"
cmake --build build --target test_ARGS="--repeat after-timeout:2"
cmake --build build --target test_ARGS="--test-load 4"
cmake --build build --target test_ARGS="--stop-time 23:59:59"
cmake --build build --target test_ARGS="--output-log test.log"
cmake --build build --target test_ARGS="--output-junit junit.xml"
cmake --build build --target test_ARGS="--output-chdir"
cmake --build build --target test_ARGS="--interactive-debug-mode 0"
cmake --build build --target test_ARGS="--no-compress-output"
cmake --build build --target test_ARGS="--test-output-size-passed 1024"
cmake --build build --target test_ARGS="--test-output-size-failed 4096"
cmake --build build --target test_ARGS="--repeat until-pass:5"
cmake --build build --target test_ARGS="--repeat after-timeout:3"
cmake --build build --target test_ARGS="--repeat until-fail:10"
cmake --build build --target test_ARGS="--repeat until-pass:2"
cmake --build build --target test_ARGS="--repeat after-timeout:1"
cmake --build build --target test_ARGS="--repeat until-fail:1"
cmake --build build --target test_ARGS="--repeat until-pass:1"
cmake --build build --target test_ARGS="--repeat after-timeout:0"
cmake --build build --target test_ARGS="--repeat until-fail:0"
cmake --build build --target test_ARGS="--repeat until-pass:0"
cmake --build build --target test_ARGS="--repeat after-timeout:10"
cmake --build build --target test_ARGS="--repeat until-fail:100"
cmake --build build --target test_ARGS="--repeat until-pass:100"
cmake --build build --target test_ARGS="--repeat after-timeout:100"
cmake --build build --target test_ARGS="--repeat until-fail:1000"
cmake --build build --target test_ARGS="--repeat until-pass:1000"
cmake --build build --target test_ARGS="--repeat after-timeout:1000"
cmake --build build --target test_ARGS="--repeat until-fail:10000"
cmake --build build --target test_ARGS="--repeat until-pass:10000"
cmake --build build --target test_ARGS="--repeat after-timeout:10000"
cmake --build build --target test_ARGS="--repeat until-fail:100000"
cmake --build build --target test_ARGS="--repeat until-pass:100000"
cmake --build build --target test_ARGS="--repeat after-timeout:100000"
cmake --build build --target test_ARGS="--repeat until-fail:1000000"
cmake --build build --target test_ARGS="--repeat until-pass:1000000"
cmake --build build --target test_ARGS="--repeat after-timeout:1000000"
cmake --build build --target test_ARGS="--repeat until-fail:10000000"
cmake --build build --target test_ARGS="--repeat until-pass:10000000"
cmake --build build --target test_ARGS="--repeat after-timeout:10000000"
cmake --build build --target test_ARGS="--repeat until-fail:100000000"
cmake --build build --target test_ARGS="--repeat until-pass:100000000"
cmake --build build --target test_ARGS="--repeat after-timeout:100000000"
cmake --build build --target test_ARGS="--repeat until-fail:1000000000"
cmake --build build --target test_ARGS="--repeat until-pass:1000000000"
cmake --build build --target test_ARGS="--repeat after-timeout:1000000000"
cmake --build build --target test_ARGS="--repeat until-fail:2147483647"
cmake --build build --target test_ARGS="--repeat until-pass:2147483647"
cmake --build build --target test_ARGS="--repeat after-timeout:2147483647"
cmake --build build --target test_ARGS="--repeat until-fail:4294967295"
cmake --build build --target test_ARGS="--repeat until-pass:4294967295"
cmake --build build --target test_ARGS="--repeat after-timeout:4294967295"
cmake --build build --target test_ARGS="--repeat until-fail:9223372036854775807"
cmake --build build --target test_ARGS="--repeat until-pass:9223372036854775807"
cmake --build build --target test_ARGS="--repeat after-timeout:9223372036854775807"
cmake --build build --target test_ARGS="--repeat until-fail:18446744073709551615"
cmake --build build --target test_ARGS="--repeat until-pass:18446744073709551615"
cmake --build build --target test_ARGS="--repeat after-timeout:18446744073709551615"
cmake --build build --target test_ARGS="--repeat until-fail:340282366920938463463374607431768211455"
cmake --build build --target test_ARGS="--repeat until-pass:340282366920938463463374607431768211455"
cmake --build build --target test_ARGS="--repeat after-timeout:340282366920938463463374607431768211455"
cmake --build build --target test_ARGS="--repeat until-fail:115792089237316195423570985008687907853269984665640564039457584007913129639935"
cmake --build build --target test_ARGS="--repeat until-pass:115792089237316195423570985008687907853269984665640564039457584007913129639935"
cmake --build build --target test_ARGS="--repeat after-timeout:115792089237316195423570985008687907853269984665640564039457584007913129639935"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084095"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084095"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084095"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900608409"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900608409"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900608409"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490060840"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490060840"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490060840"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900608"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900608"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900608"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490060"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490060"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490060"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364900"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336490"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643364"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464336"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994643"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699464"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656994"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465699"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194656"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119465"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281194"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828119"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388281"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538828"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375388"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537538"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085375"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508537"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605085"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860508"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648605"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864860"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818648"
cmake --build build --target test_ARGS="--repeat until-fail:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864"
cmake --build build --target test_ARGS="--repeat until-pass:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864"
cmake --build build --target test_ARGS="--repeat after-timeout:134078079299425970995740249982058461274793658205923933777235614437217640300735469768018742981669034276900318581864"
cmake --build build --target test_ARGS="--repeat until-fail:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186"
cmake --build build --target test_ARGS="--repeat until-pass:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186"
cmake --build build --target test_ARGS="--repeat after-timeout:13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186"
cmake --build build --target test_ARGS="--repeat until-fail:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818"
cmake --build build --target test_ARGS="--repeat until-pass:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818"
cmake --build build --target test_ARGS="--repeat after-timeout:1340780792994259709957402499820584612747936582059239337772356144372176403007354697680187429816690342769003185818
```

```
systemctl restart nginx
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
# Auto-generated by Alembic - please review before committing
# Generated: 2024-12-19 03:15:00 UTC
# Revision: 7f3a9c2e1b4d
# Down revision: 2e8b5d1f6a3c

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

# revision identifiers, used by Alembic.
revision = '7f3a9c2e1b4d'
down_revision = '2e8b5d1f6a3c'
branch_labels = None
depends_on = None


def upgrade():
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('webhook_delivery_log',
    sa.Column('id', sa.Integer(), nullable=False),
    sa.Column('webhook_id', sa.Integer(), nullable=False),
    sa.Column('event_type', sa.String(length=64), nullable=False),
    sa.Column('payload_hash', sa.String(length=128), nullable=False),
    sa.Column('response_status', sa.Integer(), nullable=True),
    sa.Column('response_body', sa.Text(), nullable=True),
    sa.Column('attempt_count', sa.Integer(), server_default='1', nullable=False),
    sa.Column('delivered_at', sa.DateTime(timezone=True), nullable=True),
    sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.ForeignKeyConstraint(['webhook_id'], ['webhook_subscription.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id')
    )
    op.create_index('ix_webhook_delivery_log_created_at', 'webhook_delivery_log', ['created_at'], unique=False)
    op.create_index('ix_webhook_delivery_log_webhook_id', 'webhook_delivery_log', ['webhook_id'], unique=False)
    # ### end Alembic commands ###


def downgrade():
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_index('ix_webhook_delivery_log_webhook_id', table_name='webhook_delivery_log')
    op.drop_index('ix_webhook_delivery_log_created_at', table_name='webhook_delivery_log')
    op.drop_table('webhook_delivery_log')
    # ### end Alembic commands ###
```

Introduction to Cooking Recipes

Cooking is an art that requires practice, patience, and a willingness to experiment. Whether you are a beginner or an experienced cook, having a collection of reliable recipes can make the difference between a stressful evening and a delightful one. This guide aims to provide you with a comprehensive collection of recipes that cover various cuisines, techniques, and skill levels.

The recipes in this guide have been carefully selected and tested to ensure that they are clear, accurate, and delicious. Each recipe includes a list of ingredients, step-by-step instructions, and helpful tips to help you achieve the best results. We have also included information about preparation time, cooking time, and the number of servings to help you plan your meals effectively.

In addition to the recipes, this guide also includes sections on kitchen basics, such as how to choose and care for your cookware, how to stock your pantry, and how to develop your own recipes. We believe that cooking is a skill that can be learned by anyone, and we hope that this guide will inspire you to spend more time in the kitchen and enjoy the process of creating delicious meals for yourself and your loved ones.

Table of Contents

- Introduction to Cooking Recipes
- Table of Contents
- Kitchen Basics
  - Choosing Your Cookware
  - Caring for Your Cookware
  - Stocking Your Pantry
  - Developing Your Own Recipes
- Breakfast Recipes
  - Classic Pancakes
  - Fluffy Omelette
  - Overnight Oats
  - Avocado Toast
  - Granola Bowl
  - Smoothie Bowl
  - Breakfast Burrito
  - Eggs Benedict
  - French Toast
  - Breakfast Sandwich
- Lunch Recipes
  - Caesar Salad
  - Grilled Cheese Sandwich
  - Tomato Soup
  - Chicken Noodle Soup
  - Quinoa Salad
  - Tuna Salad
  - Veggie Wrap
  - Caprese Salad
  - Minestrone Soup
  - Lentil Soup
- Dinner Recipes
  - Spaghetti Bolognese
  - Chicken Parmesan
  - Beef Stir-Fry
  - Vegetable Curry
  - Salmon with Roasted Vegetables
  - Shrimp Scampi
  - Pork Chops with Apple Sauce
  - Lasagna
  - Chicken Fajitas
  - Beef Tacos
- Dessert Recipes
  - Chocolate Chip Cookies
  - Brownies
  - Apple Pie
  - Cheesecake
  - Ice Cream Sundae
  - Fruit Salad
  - Banana Bread
  - Lemon Bars
  - Peanut Butter Cookies
  - Rice Pudding
- Appendix
  - Measurement Conversions
  - Glossary of Terms
  - Index

Kitchen Basics

Choosing Your Cookware

When it comes to choosing cookware, there are many options available, ranging from stainless steel to cast iron to non-stick. Each material has its own advantages and disadvantages, and the best choice for you will depend on your cooking style and preferences.

Stainless steel is a durable and versatile material that is resistant to rust and corrosion. It is also relatively easy to clean and maintain. However, it can be prone to sticking if not properly preheated and oiled.

Cast iron is a heavy and durable material that is excellent for searing and browning. It can also be used in the oven and on the stovetop. However, it requires seasoning and can be difficult to clean.

Non-stick cookware is coated with a material that prevents food from sticking. It is easy to clean and requires little oil. However, the coating can be scratched or damaged if not used properly.

Caring for Your Cookware

Proper care and maintenance of your cookware can extend its lifespan and ensure that it performs well. Here are some general guidelines for caring for different types of cookware.

Stainless steel cookware can be washed in the dishwasher or by hand with warm soapy water. Avoid using abrasive cleaners or scouring pads, as they can scratch the surface.

Cast iron cookware should be seasoned regularly to prevent rust and maintain its non-stick properties. To season cast iron, coat it with a thin layer of oil and bake it in the oven at a high temperature.

Non-stick cookware should be washed by hand with a soft sponge or cloth. Avoid using metal utensils, as they can damage the coating.

Stocking Your Pantry

A well-stocked pantry can make it easy to prepare delicious meals at a moment's notice. Here are some essential items to keep on hand.

Dry goods such as flour, sugar, rice, pasta, and beans have a long shelf life and can be used in a variety of recipes.

Canned goods such as tomatoes, tuna, and soup are convenient and can be used in a pinch.

Spices and herbs add flavor to your dishes and can be used fresh or dried.

Oils and vinegars are essential for cooking and dressing salads.

Developing Your Own Recipes

Once you have a good understanding of the basics of cooking, you can start developing your own recipes. Here are some tips to help you get started.

Start with a simple recipe that you are familiar with and make small changes to it.

Experiment with different flavors and ingredients.

Keep a record of your recipes and make notes about what you like and what you would change.

Don't be afraid to make mistakes. They are a part of the learning process.

Breakfast Recipes

Classic Pancakes

Pancakes are a delicious and easy breakfast that can be enjoyed by everyone. This recipe makes about 8 pancakes.

Ingredients:

- 1 cup all-purpose flour
- 2 tablespoons sugar
- 2 teaspoons baking powder
- 1/2 teaspoon salt
- 1 cup milk
- 1 large egg
- 2 tablespoons melted butter
- 1 teaspoon vanilla extract

Instructions:

1. In a large bowl, whisk together the flour, sugar, baking powder, and salt.
2. In a separate bowl, whisk together the milk, egg, melted butter, and vanilla extract.
3. Pour the wet ingredients into the dry ingredients and stir until just combined. Do not overmix.
4. Heat a lightly oiled griddle or frying pan over medium heat.
5. Pour about 1/4 cup of batter onto the griddle for each pancake.
6. Cook until bubbles form on the surface of the pancakes and the edges are dry, about 2-3 minutes.
7. Flip the pancakes and cook until the other side is golden brown, about 1-2 minutes.
8. Serve hot with your favorite toppings.

Fluffy Omelette

An omelette is a quick and easy breakfast that can be customized with your favorite fillings. This recipe makes one omelette.

Ingredients:

- 2 large eggs
- 1 tablespoon milk
- Salt and pepper to taste
- 1 tablespoon butter
- Fillings of your choice (cheese, vegetables, meat, etc.)

Instructions:

1. In a small bowl, whisk together the eggs, milk, salt, and pepper.
2. Heat a non-stick frying pan over medium heat.
3. Add the butter to the pan and swirl to coat.
4. Pour the egg mixture into the pan.
5. Cook until the edges begin to set, about 1 minute.
6. Add your fillings to one half of the omelette.
7. Fold the other half of the omelette over the fillings.
8. Cook until the omelette is cooked through, about 1-2 minutes.
9. Slide the omelette onto a plate and serve hot.

Overnight Oats

Overnight oats are a convenient and healthy breakfast that can be prepared the night before. This recipe makes one serving.

Ingredients:

- 1/2 cup rolled oats
- 1/2 cup milk
- 1/2 cup yogurt
- 1 tablespoon chia seeds
- 1 tablespoon maple syrup
- 1/2 teaspoon vanilla extract
- Fruit, nuts, and seeds for topping

Instructions:

1. In a jar or container with a lid, combine the oats, milk, yogurt, chia seeds, maple syrup, and vanilla extract.
2. Stir well to combine.
3. Cover and refrigerate overnight.
4. In the morning, top with fruit, nuts, and seeds.
5. Enjoy cold or heat in the microwave for a few minutes.

Avocado Toast

Avocado toast is a simple and delicious breakfast that is packed with healthy fats and nutrients. This recipe makes one serving.

Ingredients:

- 1 slice of bread
- 1/2 avocado
- Salt and pepper to taste
- Optional toppings: red pepper flakes, everything bagel seasoning, sliced tomato, egg

Instructions:

1. Toast the bread to your desired level of crispiness.
2. Mash the avocado in a bowl with salt and pepper.
3. Spread the mashed avocado on the toast.
4. Add any optional toppings.
5. Serve immediately.

Granola Bowl

A granola bowl is a customizable breakfast that can be made with your favorite granola and toppings. This recipe makes one serving.

Ingredients:

- 1 cup granola
- 1 cup yogurt
- Fruit, nuts, and seeds for topping
- Optional: honey or maple syrup

Instructions:

1. In a bowl, combine the granola and yogurt.
2. Top with fruit, nuts, and seeds.
3. Drizzle with honey or maple syrup if desired.
4. Enjoy immediately.

Smoothie Bowl

A smoothie bowl is a thick and creamy breakfast that is topped with fruit, granola, and other delicious toppings. This recipe makes one serving.

Ingredients:

- 1 frozen banana
- 1 cup frozen berries
- 1/2 cup milk
- 1/2 cup yogurt
- Optional: 1 tablespoon honey or maple syrup
- Toppings: granola, fruit, nuts, seeds, coconut flakes

Instructions:

1. In a blender, combine the banana, berries, milk, yogurt, and honey or maple syrup.
2. Blend until smooth and creamy.
3. Pour into a bowl.
4. Top with granola, fruit, nuts, seeds, and coconut flakes.
5. Enjoy immediately.

Breakfast Burrito

A breakfast burrito is a hearty and filling breakfast that is perfect for on-the-go. This recipe makes one burrito.

Ingredients:

- 1 large tortilla
- 2 large eggs, scrambled
- 1/4 cup cooked black beans
- 1/4 cup cooked rice
- 1/4 cup shredded cheese
- Salsa, sour cream, and guacamole for serving

Instructions:

1. Warm the tortilla in a dry frying pan or microwave.
2. Scramble the eggs in a separate pan.
3. Assemble the burrito by placing the eggs, black beans, rice, and cheese in the center of the tortilla.
4. Fold the sides of the tortilla over the filling, then roll up tightly.
5. Serve with salsa, sour cream, and guacamole.

Eggs Benedict

Eggs Benedict is a classic breakfast dish that consists of poached eggs, Canadian bacon, and hollandaise sauce on an English muffin. This recipe makes two servings.

Ingredients:

- 2 English muffins, split and toasted
- 4 slices Canadian bacon
- 4 large eggs
- 1 tablespoon white vinegar
- Hollandaise sauce (recipe below)

Hollandaise Sauce Ingredients:

- 3 large egg yolks
- 1 tablespoon lemon juice
- 1/2 cup melted butter
- Salt and pepper to taste

Instructions:

1. Prepare the hollandaise sauce: In a blender, combine the egg yolks and lemon juice. Blend until smooth.
2. With the blender running, slowly drizzle in the melted butter until the sauce is thick and creamy.
3. Season with salt and pepper to taste.
4. Poach the eggs: Bring a pot of water to a simmer and add the vinegar.
5. Crack each egg into a small bowl and gently slide it into the simmering water.
6. Cook for 3-4 minutes, or until the whites are set and the yolks are still runny.
7. Remove the eggs with a slotted spoon and drain on paper towels.
8. Assemble the eggs Benedict: Place a slice of Canadian bacon on each English muffin half.
9. Top with a poached egg and hollandaise sauce.
10. Serve immediately.

French Toast

French toast is a delicious breakfast that is made by dipping bread in a mixture of eggs and milk and then frying it. This recipe makes four servings.

Ingredients:

- 8 slices of bread
- 4 large eggs
- 1 cup milk
- 1 teaspoon vanilla extract
- 1/2 teaspoon cinnamon
- Butter for frying
- Maple syrup for serving

Instructions:

1. In a shallow bowl, whisk together the eggs, milk, vanilla extract, and cinnamon.
2. Heat a lightly oiled griddle or frying pan over medium heat.
3. Dip each slice of bread in the egg mixture, coating both sides.
4. Fry the bread until golden brown, about 2-3 minutes per side.
5. Serve hot with maple syrup.

Breakfast Sandwich

A breakfast sandwich is a quick and easy breakfast that can be customized with your favorite fillings. This recipe makes one sandwich.

Ingredients:

- 1 English muffin, split and toasted
- 1 large egg, fried or scrambled
- 1 slice cheese
- 1 slice bacon, cooked
- Optional: avocado, tomato, spinach

Instructions:

1. Assemble the sandwich by placing the egg, cheese, bacon, and any optional fillings on the bottom half of the English muffin.
2. Top with the other half of the English muffin.
3. Serve immediately.

Lunch Recipes

Caesar Salad

Caesar salad is a classic salad that consists of romaine lettuce, croutons, and Caesar dressing. This recipe makes four servings.

Ingredients:

- 1 head romaine lettuce, chopped
- 1 cup croutons
- 1/2 cup Caesar dressing
- Optional: grilled chicken, shrimp, or salmon

Instructions:

1. In a large bowl, combine the romaine lettuce, croutons, and Caesar dressing.
2. Toss to coat.
3. Top with grilled chicken, shrimp, or salmon if desired.
4. Serve immediately.

Grilled Cheese Sandwich

A grilled cheese sandwich is a quick and easy lunch that is perfect for any time of day. This recipe makes one sandwich.

Ingredients:

- 2 slices of bread
- 2 slices cheese
- Butter for frying

Instructions:

1. Heat a lightly oiled griddle or frying pan over medium heat.
2. Butter one side of each slice of bread.
3. Place the cheese between the slices of bread, with the buttered sides facing out.
4. Fry the sandwich until golden brown and the cheese is melted, about 2-3 minutes per side.
5. Serve hot.

Tomato Soup

Tomato soup is a comforting and delicious soup that is perfect for a cold day. This recipe makes four servings.

Ingredients:

- 1 tablespoon olive oil
- 1 onion, chopped
- 2 cloves garlic, minced
- 1 (28 ounce) can crushed tomatoes
- 1 cup vegetable broth
- 1 teaspoon dried basil
- 1 teaspoon dried oregano
- Salt and pepper to taste
- Optional: cream or milk

Instructions:

1. Heat the olive oil in a large pot over medium heat.
2. Add the onion and garlic and cook until softened, about 5 minutes.
3. Add the crushed tomatoes, vegetable broth, basil, oregano, salt, and pepper.
4. Bring to a simmer and cook for 15 minutes.
5. Use an immersion blender to puree the soup until smooth.
6. Stir in cream or milk if desired.
7. Serve hot.

Chicken Noodle Soup

Chicken noodle soup is a classic soup that is perfect for when you are feeling sick or just need a comforting meal. This recipe makes six servings.

Ingredients:

- 1 tablespoon olive oil
- 1 onion, chopped
- 2 carrots, chopped
- 2 celery stalks, chopped
- 2 cloves garlic, minced
- 6 cups chicken broth
- 1 cup cooked chicken, shredded
- 1 cup egg noodles
- 1 teaspoon dried thyme
- Salt and pepper to taste

Instructions:

1. Heat the olive oil in a large pot over medium heat.
2. Add the onion, carrots, celery, and garlic and cook until softened, about 5 minutes.
3. Add the chicken broth, chicken, egg noodles, thyme, salt, and pepper.
4. Bring to a simmer and cook for 10 minutes, or until the noodles are cooked through.
5. Serve hot.

Quinoa Salad

Quinoa salad is a healthy and delicious salad that is packed with protein and fiber. This recipe makes four servings.

Ingredients:

- 1 cup quinoa, cooked
- 1 cup chopped vegetables (cucumber, tomato, bell pepper, etc.)
- 1/4 cup chopped fresh herbs (parsley, cilantro, mint, etc.)
- 1/4 cup vinaigrette dressing
- Optional: feta cheese, olives, nuts, seeds

Instructions:

1. In a large bowl, combine the quinoa, vegetables, herbs, and vinaigrette dressing.
2. Toss to coat.
3. Top with feta cheese, olives, nuts, or seeds if desired.
4. Serve immediately or chilled.

Tuna Salad

Tuna salad is a classic lunch that is easy to make and can be enjoyed on its own or in a sandwich. This recipe makes two servings.

Ingredients:

- 1 (5 ounce) can tuna, drained
- 1/4 cup mayonnaise
- 1 tablespoon chopped celery
- 1 tablespoon chopped onion
- 1 tablespoon chopped pickles
- Salt and pepper to taste

Instructions:

1. In a bowl, combine the tuna, mayonnaise, celery, onion, pickles, salt, and pepper.
2. Mix well.
3. Serve on bread, crackers, or lettuce.

Veggie Wrap

A veggie wrap is a healthy and delicious lunch that is packed with vegetables and flavor. This recipe makes one wrap.

Ingredients:

- 1 large tortilla
- 1/4 cup hummus
- 1/2 cup chopped vegetables (lettuce, tomato, cucumber, bell pepper, etc.)
- Optional: feta cheese, olives, sprouts

Instructions:

1. Spread the hummus on the tortilla.
2. Top with the vegetables and any optional fillings.
3. Roll up the tortilla tightly.
4. Cut in half and serve.

Caprese Salad

Caprese salad is a simple and elegant salad that is perfect for a summer lunch. This recipe makes four servings.

Ingredients:

- 4 large tomatoes, sliced
- 8 ounces fresh mozzarella cheese, sliced
- 1/4 cup fresh basil leaves
- 2 tablespoons olive oil
- 1 tablespoon balsamic vinegar
- Salt and pepper to taste

Instructions:

1. Arrange the tomatoes, mozzarella cheese, and basil leaves on a platter.
2. Drizzle with olive oil and balsamic vinegar.
3. Season with salt and pepper to taste.
4. Serve immediately.

Minestrone Soup

Minestrone soup is a hearty and delicious soup that is packed with vegetables and beans. This recipe makes six servings.

Ingredients:

- 1 tablespoon olive oil
- 1 onion, chopped
- 2 carrots, chopped
- 2 celery stalks, chopped
- 2 cloves garlic, minced
- 6 cups vegetable broth
- 1 (15 ounce) can diced tomatoes
- 1 (15 ounce) can kidney beans, rinsed and drained
- 1 (15 ounce) can cannellini beans, rinsed and drained
- 1 cup chopped zucchini
- 1 cup chopped green beans
- 1 teaspoon dried basil
- 1 teaspoon dried oregano
- Salt and pepper to taste
- Optional: pasta, rice, or quinoa

Instructions:

1. Heat the olive oil in a large pot over medium heat.
2. Add the onion, carrots, celery, and garlic and cook until softened, about 5 minutes.
3. Add the vegetable broth, diced tomatoes, kidney beans, cannellini beans, zucchini, green beans, basil, oregano, salt, and pepper.
4. Bring to a simmer and cook for 15 minutes.
5. Add pasta, rice, or quinoa if desired and cook until tender.
6. Serve hot.

Lentil Soup

Lentil soup is a healthy and delicious soup that is packed with protein and fiber. This recipe makes six servings.

Ingredients:

- 1 tablespoon olive oil
- 1 onion, chopped
- 2 carrots, chopped
- 2 celery stalks, chopped
- 2 cloves garlic, minced
- 6 cups vegetable broth
- 1 cup dried lentils, rinsed and drained
- 1 (15 ounce) can diced tomatoes
- 1 teaspoon dried thyme
- 1 teaspoon dried cumin
- Salt and pepper to taste

Instructions:

1. Heat the olive oil in a large pot over medium heat.
2. Add the onion, carrots, celery, and garlic and cook until softened, about 5 minutes.
3. Add the vegetable broth, lentils, diced tomatoes, thyme, cumin, salt, and pepper.
4. Bring to a simmer and cook for 30 minutes, or until the lentils are tender.
5. Serve hot.

Dinner Recipes

Spaghetti Bolognese

Spaghetti Bolognese is a classic Italian dish that consists of spaghetti with a meat sauce. This recipe makes four servings.

Ingredients:

- 1 pound ground beef
- 1 onion, chopped
- 2 cloves garlic, minced
- 1 (28 ounce) can crushed tomatoes
- 1 cup beef broth
- 1 teaspoon dried basil
- 1 teaspoon dried oregano
- Salt and pepper to taste
- 1 pound spaghetti
- Grated Parmesan cheese for serving

Instructions:

1. In a large pot, brown the ground beef, onion, and garlic over medium heat.
2. Drain off any excess fat.
3. Add the crushed tomatoes, beef broth, basil, oregano, salt, and pepper.
4. Bring to a simmer and cook for 30 minutes, or until the sauce has thickened.
5. Cook the spaghetti according to package directions.
6. Serve the spaghetti with the Bolognese sauce and grated Parmesan cheese.

Chicken Parmesan

Chicken Parmesan is a delicious Italian-American dish that consists of breaded chicken cutlets topped with marinara sauce and mozzarella cheese. This recipe makes four servings.

Ingredients:

- 4 boneless, skinless chicken breasts
- 1 cup breadcrumbs
- 1/2 cup grated Parmesan cheese
- 1 teaspoon dried basil
- 1 teaspoon dried oregano
- Salt and pepper to taste
- 2 large eggs, beaten
- 1/4 cup all-purpose flour
- 1 cup marinara sauce
- 8 ounces mozzarella cheese, shredded
- Fresh basil leaves for garnish

Instructions:

1. Preheat oven to 400°F (200°C).
2. In a shallow bowl, combine the breadcrumbs, Parmesan cheese, basil, oregano, salt, and pepper.
3. Dredge the chicken breasts in the flour, then dip in the beaten eggs, and finally coat with the breadcrumb mixture.
4. Place the chicken breasts in a baking dish.
5. Bake for 20 minutes, or until the chicken is cooked through.
6. Top the chicken with marinara sauce and mozzarella cheese.
7. Bake for an additional 5 minutes, or until the cheese is melted and bubbly.
8. Garnish with fresh basil leaves and serve hot.

Beef Stir-Fry

Beef stir-fry is a quick and easy dinner that is packed with flavor and vegetables. This recipe makes four servings.

Ingredients:

- 1 pound beef sirloin, thinly sliced
- 2 tablespoons soy sauce
- 1 tablespoon cornstarch
- 1 tablespoon vegetable oil
- 1 onion, sliced
- 2 cloves garlic, minced
- 1 bell pepper, sliced
- 1 cup broccoli florets
- 1 cup snow peas
- 1/4 cup oyster sauce
- 1 tablespoon sesame oil
- Cooked rice for serving

Instructions:

1. In a bowl, combine the beef, soy sauce, and cornstarch. Marinate for 15 minutes.
2. Heat the vegetable oil in a large wok or frying pan over high heat.
3. Add the onion, garlic, bell pepper, broccoli, and snow peas and stir-fry for 3-4 minutes, or until the vegetables are tender-crisp.
4. Remove the vegetables from the wok and set aside.
5. Add the beef to the wok and stir-fry for 2-3 minutes, or until cooked through.
6. Return the vegetables to the wok and add the oyster sauce and sesame oil.
7. Stir-fry for 1 minute, or until heated through.
8. Serve hot with cooked rice.

Vegetable Curry

Vegetable curry is a flavorful and aromatic dish that is perfect for a vegetarian dinner. This recipe makes four servings.

Ingredients:

- 1 tablespoon vegetable oil
- 1 onion, chopped
- 2 cloves garlic, minced
- 1 tablespoon curry powder
- 1 teaspoon ground cumin
- 1 teaspoon ground coriander
- 1 (14 ounce) can coconut milk
- 1 cup vegetable broth
- 1 cup chopped vegetables (potatoes, carrots, cauliflower, etc.)
- 1 cup chickpeas, rinsed and drained
- Salt and pepper to taste
- Cooked rice for serving

Instructions:

1. Heat the vegetable oil in a large pot over medium heat.
2. Add the onion and garlic and cook until softened, about 5 minutes.
3. Add the curry powder, cumin, and coriander and cook for 1 minute, or until fragrant.
4. Add the coconut milk, vegetable broth, vegetables, chickpeas, salt, and pepper.
5. Bring to a simmer and cook for 20 minutes, or until the vegetables are tender.
6. Serve hot with cooked rice.

Salmon with Roasted Vegetables

Salmon with roasted vegetables is a healthy and delicious dinner that is packed with protein and nutrients. This recipe makes four servings.

Ingredients:

- 4 salmon fillets
- 1 tablespoon olive oil
- Salt and pepper to taste
- 1 pound asparagus, trimmed
- 1 pint cherry tomatoes
- 1 lemon, sliced
- Fresh dill for garnish

Instructions:

1. Preheat oven to 400°F (200°C).
2. Place the salmon fillets on a baking sheet lined with parchment paper.
3. Drizzle with olive oil and season with salt and pepper.
4. Arrange the asparagus and cherry tomatoes around the salmon.
5. Top the salmon with lemon slices.
6. Bake for 15-20 minutes, or until the salmon is cooked through and the vegetables are tender.
7. Garnish with fresh dill and serve hot.

Shrimp Scampi

Shrimp scampi is a delicious and elegant dish that is perfect for a special occasion. This recipe makes four servings.

Ingredients:

- 1 pound large shrimp, peeled and deveined
- 1/4 cup butter
- 2 cloves garlic, minced
- 1/4 cup white wine
- 1/4 cup lemon juice
- 1 tablespoon chopped fresh parsley
- Salt and pepper to taste
- Cooked pasta for serving

Instructions:

1. Melt the butter in a large frying pan over medium heat.
2. Add the garlic and cook for 1 minute, or until fragrant.
3. Add the shrimp and cook for 2-3 minutes, or until pink and opaque.
4. Add the white wine, lemon juice, parsley, salt, and pepper.
5. Cook for 1 minute, or until heated through.
6. Serve hot with cooked pasta.

Pork Chops with Apple Sauce

Pork chops with apple sauce is a classic dinner that is easy to make and delicious. This recipe makes four servings.

Ingredients:

- 4 pork chops
- 1 tablespoon vegetable oil
- Salt and pepper to taste
- 1 cup applesauce
- 1/2 teaspoon ground cinnamon
- 1/4 teaspoon ground nutmeg

Instructions:

1. Heat the vegetable oil in a large frying pan over medium heat.
2. Season the pork chops with salt and pepper.
3. Cook the pork chops for 3-4 minutes per side, or until cooked through.
4. Remove the pork chops from the pan and set aside.
5. Add the applesauce, cinnamon, and nutmeg to the pan and cook for 1 minute, or until heated through.
6. Serve the pork chops with the apple sauce.

Lasagna

Lasagna is a delicious and comforting dish that is perfect for a family dinner. This recipe makes eight servings.

Ingredients:

- 1 pound ground beef
- 1 onion, chopped
- 2 cloves garlic, minced
- 1 (28 ounce) can crushed tomatoes
- 1 cup beef broth
- 1 teaspoon dried basil
- 1 teaspoon dried oregano
- Salt and pepper to taste
- 12 lasagna noodles
- 15 ounces ricotta cheese
- 1 large egg
- 1/4 cup grated Parmesan cheese
- 8 ounces mozzarella cheese, shredded

Instructions:

1. Preheat oven to 375°F (190°C).
2. In a large pot, brown the ground beef, onion, and garlic over medium heat.
3. Drain off any excess fat.
4. Add the crushed tomatoes, beef broth, basil, oregano, salt, and pepper.
5. Bring to a simmer and cook for 30 minutes, or until the sauce has thickened.
6. Cook the lasagna noodles according to package directions.
7. In a bowl, combine the ricotta cheese, egg, and Parmesan cheese.
8. Spread a thin layer of sauce in the bottom of a 9x13 inch baking dish.
9. Layer with lasagna noodles, ricotta cheese mixture, sauce, and mozzarella cheese.
10. Repeat layers, ending with mozzarella cheese.
11. Bake for 45 minutes, or until the cheese is melted and bubbly.
12. Let stand for 10 minutes before serving.

Chicken Fajitas

Chicken fajitas are a quick and easy dinner that is packed with flavor and vegetables. This recipe makes four servings.

Ingredients:

- 1 pound chicken breast, thinly sliced
- 1 tablespoon vegetable oil
- 1 onion, sliced
- 1 bell pepper, sliced
- 1 teaspoon chili powder
- 1 teaspoon ground cumin
- 1 teaspoon garlic powder
- Salt and pepper to taste
- Tortillas for serving
- Optional: sour cream, guacamole, salsa, cheese

Instructions:

1. Heat the vegetable oil in a large frying pan over medium heat.
2. Add the onion and bell pepper and cook for 3-4 minutes, or until softened.
3. Add the chicken, chili powder, cumin, garlic powder, salt, and pepper.
4. Cook for 5-6 minutes, or until the chicken is cooked through.
5. Serve hot with tortillas and optional toppings.

Beef Tacos

Beef tacos are a delicious and easy dinner that is perfect for a weeknight meal. This recipe makes four servings.

Ingredients:

- 1 pound ground beef
- 1 onion, chopped
- 2 cloves garlic, minced
- 1 tablespoon chili powder
- 1 teaspoon ground cumin
- 1 teaspoon garlic powder
- Salt and pepper to taste
- Taco shells for serving
- Optional: lettuce, tomato, cheese, sour cream, salsa

Instructions:

1. In a large frying pan, brown the ground beef, onion, and garlic over medium heat.
2. Drain off any excess fat.
3. Add the chili powder, cumin, garlic powder, salt, and pepper.
4. Cook for 5 minutes, or until heated through.
5. Serve hot with taco shells and optional toppings.

Dessert Recipes

Chocolate Chip Cookies

Chocolate chip cookies are a classic dessert that is loved by everyone. This recipe makes about 24 cookies.

Ingredients:

- 1 cup (2 sticks) butter, softened
- 1 cup granulated sugar
- 1 cup packed brown sugar
- 2 large eggs
- 1 teaspoon vanilla extract
- 3 cups all-purpose flour
- 1 teaspoon baking soda
- 1 teaspoon salt
- 2 cups chocolate chips

Instructions:

1. Preheat oven to 375°F (190°C).
2. In a large bowl, cream together the butter, granulated sugar, and brown sugar until light and fluffy.
3. Beat in the eggs and vanilla extract.
4. In a separate bowl, combine the flour, baking soda, and salt.
5. Gradually add the dry ingredients to the wet ingredients, mixing until just combined.
6. Stir in the chocolate chips.
7. Drop by rounded tablespoons onto ungreased baking sheets.
8. Bake for 9-11 minutes, or until golden brown.
9. Let cool on baking sheets for 2 minutes before transferring to wire racks to cool completely.

Brownies

Brownies are a delicious and decadent dessert that is perfect for a special occasion. This recipe makes 16 brownies.

Ingredients:

- 1 cup (2 sticks) butter, melted
- 2 cups granulated sugar
- 4 large eggs
- 1 teaspoon vanilla extract
- 1 cup all-purpose flour
- 1/2 cup unsweetened cocoa powder
- 1/2 teaspoon salt
- Optional: chocolate chips, nuts

Instructions:

1. Preheat oven to 350°F (175°C).
2. Grease and flour a 9x13 inch baking pan.
3. In a large bowl, combine the melted butter, granulated sugar, eggs, and vanilla extract.
4. In a separate bowl, combine the flour, cocoa powder, and salt.
5. Gradually add the dry ingredients to the wet ingredients, mixing until just combined.
6. Stir in chocolate chips or nuts if desired.
7. Pour the batter into the prepared pan.
8. Bake for 25-30 minutes, or until a toothpick inserted into the center comes out with moist crumbs.
9. Let cool completely in the pan before cutting into squares.

Apple Pie

Apple pie is a classic dessert that is perfect for the fall season. This recipe makes one 9-inch pie.

Ingredients:

- 1 recipe pie crust (recipe below)
- 6 cups thinly sliced apples
- 1/2 cup granulated sugar
- 1/4 cup packed brown sugar
- 1 tablespoon all-purpose flour
- 1 teaspoon ground cinnamon
- 1/4 teaspoon ground nutmeg
- 1 tablespoon butter, cut into small pieces

Pie Crust Ingredients:

- 2 cups all-purpose flour
- 1 teaspoon salt
- 1 cup (2 sticks) cold butter, cut into small pieces
- 6-8 tablespoons ice water

Instructions:

1. Prepare the pie crust: In a large bowl, combine the flour and salt.
2. Cut in the butter until the mixture resembles coarse crumbs.
3. Add the ice water, one tablespoon at a time, until the dough comes together.
4. Divide the dough in half and form into two disks.
5. Wrap each disk in plastic wrap and refrigerate for at least 30 minutes.
6. Preheat oven to 375°F (190°C).
7. Roll out one disk of dough on a lightly floured surface and fit into a 9-inch pie plate.
8. In a large bowl, combine the apples, granulated sugar, brown sugar, flour, cinnamon, and nutmeg.
9. Pour the apple mixture into the pie crust.
10. Dot with butter.
11. Roll out the second disk of dough and place over the apple mixture.
12. Trim the edges and crimp to seal.
13. Cut slits in the top crust to allow steam to escape.
14. Bake for 45-50 minutes, or until golden brown and the apples are tender.
15. Let cool completely before serving.

Cheesecake

Cheesecake is a delicious and creamy dessert that is perfect for a special occasion. This recipe makes one 9-inch cheesecake.

Ingredients:

- 1 cup graham cracker crumbs
- 1/4 cup granulated sugar
- 1/4 cup (1/2 stick) butter, melted
- 24 ounces cream cheese, softened
- 1 cup granulated sugar
- 1 teaspoon vanilla extract
- 3 large eggs
- Optional: fruit, chocolate sauce, whipped cream

Instructions:

1. Preheat oven to 325°F (160°C).
2. In a bowl, combine the graham cracker crumbs, granulated sugar, and melted butter.
3. Press the mixture into the bottom of a 9-inch springform pan.
4. In a large bowl, beat the cream cheese until smooth.
5. Add the granulated sugar and vanilla extract and beat until combined.
6. Add the eggs one at a time, beating well after each addition.
7. Pour the cream cheese mixture into the crust.
8. Bake for 50-60 minutes, or until the center is set.
9. Let cool completely in the pan before removing the sides.
10. Refrigerate for at least 4 hours, or overnight, before serving.
11. Top with fruit, chocolate sauce, or whipped cream if desired.

Ice Cream Sundae

An ice cream sundae is a delicious and customizable dessert that is perfect for any time of day. This recipe makes one sundae.

Ingredients:

- 2 scoops ice cream
- Toppings: chocolate sauce, caramel sauce, whipped cream, nuts, sprinkles, cherry

Instructions:

1. Place the ice cream in a bowl.
2. Top with your favorite toppings.
3. Serve immediately.

Fruit Salad

Fruit salad is a healthy and refreshing dessert that is perfect for a summer day. This recipe makes four servings.

Ingredients:

- 1 cup chopped strawberries
- 1 cup chopped blueberries
- 1 cup chopped raspberries
- 1 cup chopped grapes
- 1 cup chopped melon
- 1 tablespoon honey
- 1 tablespoon lemon juice
- Optional: mint leaves for garnish

Instructions:

1. In a large bowl, combine the strawberries, blueberries, raspberries, grapes, and melon.
2. In a small bowl, whisk together the honey and lemon juice.
3. Pour the honey mixture over the fruit and toss to coat.
4. Garnish with mint leaves if desired.
5. Serve immediately or chilled.

Banana Bread

Banana bread is a delicious and moist quick bread that is perfect for breakfast or dessert. This recipe makes one loaf.

Ingredients:

- 1 cup mashed ripe bananas (about 2 bananas)
- 1/2 cup granulated sugar
- 1/2 cup packed brown sugar
- 1/2 cup (1 stick) butter, melted
- 1 large egg
- 1 teaspoon vanilla extract
- 1 teaspoon baking soda
- 1/2 teaspoon salt
- 1 1/2 cups all-purpose flour
- Optional: chocolate chips, nuts

Instructions:

1. Preheat oven to 350°F (175°C).
2. Grease and flour a 9x5 inch loaf pan.
3. In a large bowl, combine the mashed bananas, granulated sugar, brown sugar, melted butter, egg, and vanilla extract.
4. In a separate bowl, combine the baking soda, salt, and flour.
5. Gradually add the dry ingredients to the wet ingredients, mixing until just combined.
6. Stir in chocolate chips or nuts if desired.
7. Pour the batter into the prepared pan.
8. Bake for 50-60 minutes, or until a toothpick inserted into the center comes out with moist crumbs.
9. Let cool in the pan for 10 minutes before transferring to a wire rack to cool completely.

Lemon Bars

Lemon bars are a delicious and tangy dessert that is perfect for a summer day. This recipe makes 16 bars.

Ingredients:

- 1 cup (2 sticks) butter, softened
- 1/2 cup granulated sugar
- 2 cups all-purpose flour
- 4 large eggs
- 2 cups granulated sugar
- 1/2 cup lemon juice
- 1 tablespoon lemon zest
- 1/4 teaspoon baking powder
- Powdered sugar for dusting

Instructions:

1. Preheat oven to 350°F (175°C).
2. In a bowl, cream together the butter and 1/2 cup granulated sugar.
3. Add the flour and mix until combined.
4. Press the mixture into the bottom of a 9x13 inch baking pan.
5. Bake for 15-20 minutes, or until golden brown.
6. In a large bowl, whisk together the eggs, 2 cups granulated sugar, lemon juice, lemon zest, and baking powder.
7. Pour the lemon mixture over the crust.
8. Bake for 20-25 minutes, or until the center is set.
9. Let cool completely in the pan before cutting into bars.
10. Dust with powdered sugar before serving.

Peanut Butter Cookies

Peanut butter cookies are a classic dessert that is loved by everyone. This recipe makes about 24 cookies.

Ingredients:

- 1 cup peanut butter
- 1 cup granulated sugar
- 1 large egg
- 1 teaspoon vanilla extract
- 1 teaspoon baking soda
- 1/2 teaspoon salt

Instructions:

1. Preheat oven to 350°F (175°C).
2. In a large bowl, combine the peanut butter, granulated sugar, egg, and vanilla extract.
3. In a separate bowl, combine the baking soda and salt.
4. Gradually add the dry ingredients to the wet ingredients, mixing until just combined.
5. Roll the dough into balls and place on ungreased baking sheets.
6. Flatten with a fork in a crisscross pattern.
7. Bake for 8-10 minutes, or until golden brown.
8. Let cool on baking sheets for 2 minutes before transferring to wire racks to cool completely.

Rice Pudding

Rice pudding is a creamy and comforting dessert that is perfect for a cold day. This recipe makes four servings.

Ingredients:

- 1 cup cooked rice
- 2 cups milk
- 1/2 cup granulated sugar
- 1 teaspoon vanilla extract
- 1/4 teaspoon ground cinnamon
- Optional: raisins, nuts

Instructions:

1. In a saucepan, combine the cooked rice, milk, granulated sugar, vanilla extract, and cinnamon.
2. Bring to a simmer over medium heat, stirring occasionally.
3. Cook for 15-20 minutes, or until the rice is tender and the pudding has thickened.
4. Stir in raisins or nuts if desired.
5. Serve warm or chilled.

Appendix

Measurement Conversions

Volume	Metric
1 teaspoon	5 milliliters
1 tablespoon	15 milliliters
1 fluid ounce	30 milliliters
1 cup	240 milliliters
1 pint

```
}
}
}
```

```
# Auto-generated
# Do not hand-edit unless you're certain
# Generated on Sep 28 2020 13:32:43 by pkg-res 1.9.1

JFIF      ,,  C 		
  $.' ",#(7),01444'9=82<.342C  2!!22222222222222222222222222222222222222  " }  !1AQa"q2#BR$3br	
%&'()*456789:CDEFGHIJSTUVWXYZcdefghijstuvwxyz
                                       w !1AQaq"2B	#3Rbr
$4%&'()*56789:CDEFGHIJSTUVWXYZcdefghijstuvwxyz
   ? ((((((
```

```
# Auto-generated config.py
BOT_NAME = 'scrapybot'
SPIDER_MODULES = ['scrapybot.spiders']
NEWSPIDER_MODULE = 'scrapybot.spiders'
DEFAULT_ITEM_CLASS = 'scrapybot.items.ScrapybotItem'
USER_AGENT = 'scrapybot (+http://www.yourdomain.com)'
# Obey robots.txt rules
ROBOTSTXT_OBEY = True
# Configure maximum concurrent requests performed by Scrapy (default: 16)
#CONCURRENT_REQUESTS = 32
# Configure a delay for requests for the same website (default: 0)
# See http://scrapy.readthedocs.org/en/latest/topics/settings.html#download-delay
# See also autothrottle settings and docs
#DOWNLOAD_DELAY = 3
# The download delay setting will honor only one of:
#CONCURRENT_REQUESTS_PER_DOMAIN = 16
#CONCURRENT_REQUESTS_PER_IP = 16
# Disable cookies (enabled by default)
#COOKIES_ENABLED = False
# Disable Telnet Console (enabled by default)
#TELNETCONSOLE_ENABLED = False
# Override the default request headers:
#DEFAULT_REQUEST_HEADERS = {
#   'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
#   'Accept-Language': 'en',
#}
# Enable or disable spider middlewares
# See http://scrapy.readthedocs.org/en/latest/topics/spider-middleware.html
#SPIDER_MIDDLEWARES = {
#    'scrapybot.middlewares.MyCustomSpiderMiddleware': 543,
#}
# Enable or disable downloader middlewares
# See http://scrapy.readthedocs.org/en/latest/topics/downloader-middleware.html
#DOWNLOADER_MIDDLEWARES = {
#    'scrapybot.middlewares.MyCustomDownloaderMiddleware': 543,
#}
# Enable or disable extensions
# See http://scrapy.readthedocs.org/en/latest/topics/extensions.html
#EXTENSIONS = {
#    'scrapy.telnet.TelnetConsole': None,
#}
# Configure item pipelines
# See http://scrapy.readthedocs.org/en/latest/topics/item-pipeline.html
#ITEM_PIPELINES = {
#    'scrapybot.pipelines.SomePipeline': 300,
#}
# Enable and configure the AutoThrottle extension (disabled by default)
# See http://doc.scrapy.org/en/latest/topics/autothrottle.html
# NOTE: AutoThrottle will honor the standard settings for concurrency and delay
#AUTOTHROTTLE_ENABLED = True
# The initial download delay
#AUTOTHROTTLE_START_DELAY = 5
# The maximum download delay to be set in case of high latencies
#AUTOTHROTTLE_MAX_DELAY = 60
# Enable showing throttling stats for every response received:
#AUTOTHROTTLE_DEBUG = False
# Enable and configure HTTP caching (disabled by default)
# See http://scrapy.readthedocs.org/en/latest/topics/downloader-middleware.html#httpcache-middleware-settings
#HTTPCACHE_ENABLED = True
#HTTPCACHE_EXPIRATION_SECS = 0
#HTTPCACHE_DIR = 'httpcache'
#HTTPCACHE_IGNORE_HTTP_CODES = []
#HTTPCACHE_STORAGE = 'scrapy.extensions.httpcache.FilesystemCacheStorage'
```

```
static int migrate_v(sqlite3 *db, int target_version)
{
    int current_version = 0;
    sqlite3_stmt *stmt = NULL;
    int rc;

    rc = sqlite3_prepare_v2(db, "PRAGMA user_version;", -1, &stmt, NULL);
    if (rc != SQLITE_OK) return rc;
    if (sqlite3_step(stmt) == SQLITE_ROW) current_version = sqlite3_column_int(stmt, 0);
    sqlite3_finalize(stmt);

    if (current_version >= target_version) return SQLITE_OK;

    for (int v = current_version + 1; v <= target_version; v++) {
        const char *sql = migrations[v];
        if (!sql) {
            fprintf(stderr, "migrate_v: no migration for version %d\n", v);
            return SQLITE_ERROR;
        }
        char *errmsg = NULL;
        rc = sqlite3_exec(db, sql, NULL, NULL, &errmsg);
        if (rc != SQLITE_OK) {
            fprintf(stderr, "migrate_v: failed at v%d: %s\n", v, errmsg);
            sqlite3_free(errmsg);
            return rc;
        }
        char pragma[64];
        snprintf(pragma, sizeof(pragma), "PRAGMA user_version = %d;", v);
        rc = sqlite3_exec(db, pragma, NULL, NULL, NULL);
        if (rc != SQLITE_OK) return rc;
    }
    return SQLITE_OK;
}
```

```
package main

import (
    "context"
    "database/sql"
    "fmt"
    "log"
    "os"
    "time"

    _ "github.com/lib/pq"
)

func main() {
    dsn := os.Getenv("DATABASE_URL")
    if dsn == "" {
        log.Fatal("DATABASE_URL is not set")
    }

    db, err := sql.Open("postgres", dsn)
    if err != nil {
        log.Fatalf("open: %v", err)
    }
    defer db.Close()

    db.SetMaxOpenConns(25)
    db.SetMaxIdleConns(5)
    db.SetConnMaxLifetime(5 * time.Minute)

    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    if err := db.PingContext(ctx); err != nil {
        log.Fatalf("ping: %v", err)
    }

    rows, err := db.QueryContext(ctx, `
        SELECT id, name, email, created_at
        FROM users
        WHERE active = true
        ORDER BY created_at DESC
        LIMIT 100
    `)
    if err != nil {
        log.Fatalf("query: %v", err)
    }
    defer rows.Close()

    for rows.Next() {
        var id int64
        var name, email string
        var created time.Time
        if err := rows.Scan(&id, &name, &email, &created); err != nil {
            log.Fatalf("scan: %v", err)
        }
        fmt.Printf("%d\t%s\t%s\t%s\n", id, name, email, created.Format(time.RFC3339))
    }
    if err := rows.Err(); err != nil {
        log.Fatalf("rows: %v", err)
    }
}
```

```
def test_handle_matching():
    """
    Test that a matching request returns the right response and doesn't log a warning.
    """
    with LoggingMiddleware() as middleware:
        request = HttpRequest("POST")
        request.path = "/hello/"
        request.GET = {}
        request.POST = {}
        response = middleware(request)
        assert response.status_code == 200
        assert response.content == b"Hello, world."
    assert middleware.messages == []

def test_send_default_arguments_deserialized_as_empty_dicts():
    with LoggingMiddleware(logging.WARNING) as middleware:
        request = HttpRequest("POST")
        request.path = "/hello/"
        request.GET = {"foo": "bar"}
        request.POST = {"confirm": "1"}
        response = middleware(request)
        assert response.status_code == 200
        assert response.content == b"Hello, world."
    assert middleware.messages == []

def test_serialize_datetime():
    event = {"timestamp": datetime(2021, 1, 1)}

    rv = json.dumps(event, default=inspect.serialize)

    assert rv == '{"timestamp":"2021-01-01T00:00:00Z"}'

def test_custom_repr():
    class A:
        def __repr__(self):
            return "<A instance at 0xdeadbeef>"

    class B:
        def __repr__(self):
            return "<B instance at 0xdeadbeef>"

    events = capture_events(A(), B())

    assert events == ["<A instance at 0x2a45067>", "<B instance at 0x2a45067>"]

def test_list_dir(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")

    result = runner.invoke(cli, ["list_dir", "."])

    assert result.exit_code == 0
    assert result.output == "bar\nbaz\nfoo\n"

def test_list_dir_with_dot_files(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")

    result = runner.invoke(cli, ["list_dir", "--all", "."])

    assert result.exit_code == 0
    assert result.output == ".hidden\n__pycache__\nbar\nbaz\nfoo\n"

def test_list_dir_with_long_flag(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")

    result = runner.invoke(cli, ["list_dir", "--long", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n"

def test_list_dir_with_long_flag_and_dot_files(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  .hidden\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  .hidden\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  .hidden\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  .hidden\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n-rw-r--r--  file.txt\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  .hidden\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n-rw-r--r--  file.txt\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  .hidden\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n-rw-r--r--  file.txt\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  .hidden\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  bar\ndrwxr-xr-x  baz\ndrwxr-xr-x  foo\n-rw-r--r--  file.txt\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "."])

    assert result.exit_code == 0
    assert result.output == "-rw-r--r--  file.txt\ndrwxr-xr-x  foo\ndrwxr-xr-x  baz\ndrwxr-xr-x  bar\ndrwxr-xr-x  __pycache__\ndrwxr-xr-x  .hidden\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "."])

    assert result.exit_code == 0
    assert result.output == "-rw-r--r--  file.txt\ndrwxr-xr-x  foo\ndrwxr-xr-x  baz\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  baz\ndrwxr-xr-x  bar\ndrwxr-xr-x  __pycache__\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  baz\ndrwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst_and_average(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "--average", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst_and_average_and_median(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "--average", "--median", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst_and_average_and_median_and_mode(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "--average", "--median", "--mode", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst_and_average_and_median_and_mode_and_range(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "--average", "--median", "--mode", "--range", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst_and_average_and_median_and_mode_and_range_and_variance(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "--average", "--median", "--mode", "--range", "--variance", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst_and_average_and_median_and_mode_and_range_and_variance_and_standard_deviation(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "--average", "--median", "--mode", "--range", "--variance", "--standard-deviation", "."])

    assert result.exit_code == 0
    assert result.output == "drwxr-xr-x  bar\n"

def test_list_dir_with_long_flag_and_dot_files_and_recursive_and_depth_and_file_and_size_and_human_readable_and_sort_and_reverse_and_limit_and_offset_and_filter_and_exclude_and_include_and_ignore_case_and_follow_symlinks_and_no_color_and_quiet_and_verbose_and_version_and_help_and_extra_and_more_and_less_and_most_and_least_and_best_and_worst_and_average_and_median_and_mode_and_range_and_variance_and_standard_deviation_and_skew(tmpdir, monkeypatch):
    monkeypatch.chdir(tmpdir)
    tmpdir.mkdir("foo")
    tmpdir.mkdir("bar")
    tmpdir.mkdir("baz")
    tmpdir.mkdir(".hidden")
    tmpdir.mkdir("__pycache__")
    tmpdir.join("file.txt").write("hello")

    result = runner.invoke(cli, ["list_dir", "--long", "--all", "--recursive", "--depth", "1", "--size", "--human-readable", "--sort", "name", "--reverse", "--limit", "3", "--offset", "2", "--filter", "b*", "--exclude", "*z", "--include", "bar", "--ignore-case", "--follow-symlinks", "--no-color", "--quiet", "--verbose", "--version", "--help", "--extra", "--more", "--less", "--most", "--least", "--best", "--worst", "--average", "--median", "--mode", "--range", "--variance", "--standard-deviation", "--skew", "."])

    ass
```

```
fn main() -> Result<()> {
    let cli = Cli::parse();
    match cli.command {
        Commands::Run { file } => run(file),
        Commands::Repl => repl(),
    }
}

fn run(file: Option<PathBuf>) -> Result<()> {
    let source = match file {
        Some(path) => std::fs::read_to_string(path)?,
        None => {
            let mut buf = String::new();
            std::io::stdin().read_to_string(&mut buf)?;
            buf
        }
    };
    let tokens = lex(&source)?;
    let ast = parse(tokens)?;
    let value = eval(&ast)?;
    println!("{value}");
    Ok(())
}

fn repl() -> Result<()> {
    let mut rl = rustyline::DefaultEditor::new()?;
    loop {
        match rl.readline("> ") {
            Ok(line) => {
                rl.add_history_entry(&line)?;
                match lex(&line).and_then(parse).and_then(|a| eval(&a)) {
                    Ok(v) => println!("{v}"),
                    Err(e) => eprintln!("error: {e}"),
                }
            }
            Err(rustyline::error::ReadlineError::Eof) => break,
            Err(e) => return Err(e.into()),
        }
    }
    Ok(())
}

fn lex(src: &str) -> Result<Vec<Token>> {
    let mut out = Vec::new();
    let mut chars = src.chars().peekable();
    while let Some(&c) = chars.peek() {
        match c {
            ' ' | '\t' | '\n' => { chars.next(); }
            '0'..='9' => {
                let mut n = String::new();
                while let Some(&d) = chars.peek() {
                    if d.is_ascii_digit() { n.push(d); chars.next(); } else { break; }
                }
                out.push(Token::Num(n.parse()?));
            }
            '+' => { out.push(Token::Plus); chars.next(); }
            '-' => { out.push(Token::Minus); chars.next(); }
            '*' => { out.push(Token::Star); chars.next(); }
            '/' => { out.push(Token::Slash); chars.next(); }
            '(' => { out.push(Token::LParen); chars.next(); }
            ')' => { out.push(Token::RParen); chars.next(); }
            _ => bail!("unexpected character: {c}"),
        }
    }
    Ok(out)
}

fn parse(tokens: Vec<Token>) -> Result<Expr> {
    let mut p = Parser { tokens, pos: 0 };
    let e = p.expr()?;
    if p.pos != p.tokens.len() { bail!("trailing tokens"); }
    Ok(e)
}

struct Parser { tokens: Vec<Token>, pos: usize }

impl Parser {
    fn peek(&self) -> Option<&Token> { self.tokens.get(self.pos) }
    fn bump(&mut self) -> Option<Token> { let t = self.tokens.get(self.pos).cloned(); self.pos += 1; t }
    fn expr(&mut self) -> Result<Expr> {
        let mut lhs = self.term()?;
        while matches!(self.peek(), Some(Token::Plus | Token::Minus)) {
            let op = self.bump().unwrap();
            let rhs = self.term()?;
            lhs = Expr::Bin(Box::new(lhs), op, Box::new(rhs));
        }
        Ok(lhs)
    }
    fn term(&mut self) -> Result<Expr> {
        let mut lhs = self.atom()?;
        while matches!(self.peek(), Some(Token::Star | Token::Slash)) {
            let op = self.bump().unwrap();
            let rhs = self.atom()?;
            lhs = Expr::Bin(Box::new(lhs), op, Box::new(rhs));
        }
        Ok(lhs)
    }
    fn atom(&mut self) -> Result<Expr> {
        match self.bump() {
            Some(Token::Num(n)) => Ok(Expr::Num(n)),
            Some(Token::LParen) => {
                let e = self.expr()?;
                match self.bump() { Some(Token::RParen) => Ok(e), _ => bail!("expected )") }
            }
            _ => bail!("expected number or ("),
        }
    }
}

fn eval(e: &Expr) -> Result<f64> {
    Ok(match e {
        Expr::Num(n) => *n,
        Expr::Bin(l, op, r) => {
            let (l, r) = (eval(l)?, eval(r)?);
            match op { Token::Plus => l + r, Token::Minus => l - r, Token::Star => l * r, Token::Slash => l / r, _ => unreachable!() }
        }
    })
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test] fn precedence() { assert_eq!(eval(&parse(lex("1+2*3").unwrap()).unwrap()).unwrap(), 7.0); }
    #[test] fn parens() { assert_eq!(eval(&parse(lex("(1+2)*3").unwrap()).unwrap()).unwrap(), 9.0); }
}

fn load_user_data(path: &str) -> Result<Vec<User>> {
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let users: Vec<User> = serde_json::from_str(&contents)?;
    Ok(users)
}

fn save_user_data(path: &str, users: &[User]) -> Result<()> {
    let contents = serde_json::to_string_pretty(users)?;
    let mut file = File::create(path)?;
    file.write_all(contents.as_bytes())?;
    Ok(())
}

fn find_user_by_email<'a>(users: &'a [User], email: &str) -> Option<&'a User> {
    users.iter().find(|u| u.email == email)
}

fn find_user_by_email_mut<'a>(users: &'a mut [User], email: &str) -> Option<&'a mut User> {
    users.iter_mut().find(|u| u.email == email)
}

fn next_user_id(users: &[User]) -> u64 {
    users.iter().map(|u| u.id).max().unwrap_or(0) + 1
}

fn add_user(path: &str, name: String, email: String) -> Result<User> {
    let mut users = load_user_data(path)?;
    if find_user_by_email(&users, &email).is_some() {
        return Err(anyhow!("user with email {email} already exists"));
    }
    let user = User { id: next_user_id(&users), name, email, active: true };
    users.push(user.clone());
    save_user_data(path, &users)?;
    Ok(user)
}

fn deactivate_user(path: &str, email: &str) -> Result<()> {
    let mut users = load_user_data(path)?;
    let user = find_user_by_email_mut(&mut users, email)
        .ok_or_else(|| anyhow!("no user with email {email}"))?;
    user.active = false;
    save_user_data(path, &users)?;
    Ok(())
}

fn list_active_users(path: &str) -> Result<Vec<User>> {
    let users = load_user_data(path)?;
    Ok(users.into_iter().filter(|u| u.active).collect())
}

fn export_user_data(path: &str, out: &str) -> Result<()> {
    let users = load_user_data(path)?;
    let contents = serde_json::to_string_pretty(&users)?;
    let mut file = File::create(out)?;
    file.write_all(contents.as_bytes())?;
    Ok(())
}

fn import_user_data(path: &str, src: &str) -> Result<usize> {
    let mut file = File::open(src)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let incoming: Vec<User> = serde_json::from_str(&contents)?;
    let mut users = load_user_data(path)?;
    let mut added = 0;
    for u in incoming {
        if find_user_by_email(&users, &u.email).is_none() {
            users.push(u);
            added += 1;
        }
    }
    save_user_data(path, &users)?;
    Ok(added)
}

fn count_users(path: &str) -> Result<usize> {
    Ok(load_user_data(path)?.len())
}

fn count_active_users(path: &str) -> Result<usize> {
    Ok(list_active_users(path)?.len())
}

fn reactivate_user(path: &str, email: &str) -> Result<()> {
    let mut users = load_user_data(path)?;
    let user = find_user_by_email_mut(&mut users, email)
        .ok_or_else(|| anyhow!("no user with email {email}"))?;
    user.active = true;
    save_user_data(path, &users)?;
    Ok(())
}

fn rename_user(path: &str, email: &str, new_name: String) -> Result<()> {
    let mut users = load_user_data(path)?;
    let user = find_user_by_email_mut(&mut users, email)
        .ok_or_else(|| anyhow!("no user with email {email}"))?;
    user.name = new_name;
    save_user_data(path, &users)?;
    Ok(())
}

fn change_user_email(path: &str, email: &str, new_email: String) -> Result<()> {
    let mut users = load_user_data(path)?;
    if find_user_by_email(&users, &new_email).is_some() {
        return Err(anyhow!("user with email {new_email} already exists"));
    }
    let user = find_user_by_email_mut(&mut users, email)
        .ok_or_else(|| anyhow!("no user with email {email}"))?;
    user.email = new_email;
    save_user_data(path, &users)?;
    Ok(())
}

fn remove_user(path: &str, email: &str) -> Result<()> {
    let mut users = load_user_data(path)?;
    let before = users.len();
    users.retain(|u| u.email != email);
    if users.len() == before {
        return Err(anyhow!("no user with email {email}"));
    }
    save_user_data(path, &users)?;
    Ok(())
}

fn clear_user_data(path: &str) -> Result<()> {
    save_user_data(path, &[])?;
    Ok(())
}

fn backup_user_data(path: &str) -> Result<String> {
    let backup = format!("{path}.bak");
    std::fs::copy(path, &backup)?;
    Ok(backup)
}

fn restore_user_data(path: &str, backup: &str) -> Result<()> {
    std::fs::copy(backup, path)?;
    Ok(())
}

fn user_data_exists(path: &str) -> bool {
    std::path::Path::new(path).exists()
}

fn ensure_user_data(path: &str) -> Result<()> {
    if !user_data_exists(path) {
        save_user_data(path, &[])?;
    }
    Ok(())
}

fn sorted_users_by_name(path: &str) -> Result<Vec<User>> {
    let mut users = load_user_data(path)?;
    users.sort_by(|a, b| a.name.cmp(&b.name));
    Ok(users)
}

fn sorted_users_by_id(path: &str) -> Result<Vec<User>> {
    let mut users = load_user_data(path)?;
    users.sort_by_key(|u| u.id);
    Ok(users)
}

fn search_users(path: &str, query: &str) -> Result<Vec<User>> {
    let q = query.to_lowercase();
    let users = load_user_data(path)?;
    Ok(users.into_iter()
        .filter(|u| u.name.to_lowercase().contains(&q) || u.email.to_lowercase().contains(&q))
        .collect())
}

fn user_data_summary(path: &str) -> Result<String> {
    let users = load_user_data(path)?;
    let active = users.iter().filter(|u| u.active).count();
    Ok(format!("{} users ({} active, {} inactive)", users.len(), active, users.len() - active))
}

fn merge_user_data(path: &str, other: &str) -> Result<usize> {
    let mut file = File::open(other)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let incoming: Vec<User> = serde_json::from_str(&contents)?;
    let mut users = load_user_data(path)?;
    let mut added = 0;
    for u in incoming {
        if find_user_by_email(&users, &u.email).is_none() {
            users.push(u);
            added += 1;
        }
    }
    save_user_data(path, &users)?;
    Ok(added)
}

fn validate_user_data(path: &str) -> Result<()> {
    let users = load_user_data(path)?;
    let mut seen = std::collections::HashSet::new();
    for u in &users {
        if !seen.insert(u.email.clone()) {
            return Err(anyhow!("duplicate email: {}", u.email));
        }
    }
    Ok(())
}

fn user_data_path() -> String {
    std::env::var("USER_DATA_PATH").unwrap_or_else(|_| "users.json".to_string())
}

fn with_user_data<F, T>(f: F) -> Result<T>
where
    F: FnOnce(&str) -> Result<T>,
{
    let path = user_data_path();
    ensure_user_data(&path)?;
    f(&path)
}

fn add_user_default(name: String, email: String) -> Result<User> {
    with_user_data(|p| add_user(p, name, email))
}

fn deactivate_user_default(email: &str) -> Result<()> {
    with_user_data(|p| deactivate_user(p, email))
}

fn list_active_users_default() -> Result<Vec<User>> {
    with_user_data(|p| list_active_users(p))
}

fn count_users_default() -> Result<usize> {
    with_user_data(|p| count_users(p))
}

fn search_users_default(query: &str) -> Result<Vec<User>> {
    with_user_data(|p| search_users(p, query))
}

fn user_data_summary_default() -> Result<String> {
    with_user_data(|p| user_data_summary(p))
}

fn backup_user_data_default() -> Result<String> {
    with_user_data(|p| backup_user_data(p))
}

fn validate_user_data_default() -> Result<()> {
    with_user_data(|p| validate_user_data(p))
}

fn sorted_users_by_name_default() -> Result<Vec<User>> {
    with_user_data(|p| sorted_users_by_name(p))
}

fn sorted_users_by_id_default() -> Result<Vec<User>> {
    with_user_data(|p| sorted_users_by_id(p))
}

fn reactivate_user_default(email: &str) -> Result<()> {
    with_user_data(|p| reactivate_user(p, email))
}

fn rename_user_default(email: &str, new_name: String) -> Result<()> {
    with_user_data(|p| rename_user(p, email, new_name))
}

fn change_user_email_default(email: &str, new_email: String) -> Result<()> {
    with_user_data(|p| change_user_email(p, email, new_email))
}

fn remove_user_default(email: &str) -> Result<()> {
    with_user_data(|p| remove_user(p, email))
}

fn clear_user_data_default() -> Result<()> {
    with_user_data(|p| clear_user_data(p))
}

fn export_user_data_default(out: &str) -> Result<()> {
    with_user_data(|p| export_user_data(p, out))
}

fn import_user_data_default(src: &str) -> Result<usize> {
    with_user_data(|p| import_user_data(p, src))
}

fn merge_user_data_default(other: &str) -> Result<usize> {
    with_user_data(|p| merge_user_data(p, other))
}

fn restore_user_data_default(backup: &str) -> Result<()> {
    with_user_data(|p| restore_user_data(p, backup))
}

fn user_data_exists_default() -> bool {
    user_data_path().as_str().pipe(|p| std::path::Path::new(p).exists())
}

fn ensure_user_data_default() -> Result<()> {
    with_user_data(|_| Ok(()))
}

fn load_user_data_default() -> Result<Vec<User>> {
    with_user_data(|p| load_user_data(p))
}

fn save_user_data_default(users: &[User]) -> Result<()> {
    with_user_data(|p| save_user_data(p, users))
}

fn add_user_default_path(name: String, email: String) -> Result<User> {
    add_user_default(name, email)
}

fn deactivate_user_default_path(email: &str) -> Result<()> {
    deactivate_user_default(email)
}

fn list_active_users_default_path() -> Result<Vec<User>> {
    list_active_users_default()
}

fn count_users_default_path() -> Result<usize> {
    count_users_default()
}

fn search_users_default_path(query: &str) -> Result<Vec<User>> {
    search_users_default(query)
}

fn user_data_summary_default_path() -> Result<String> {
    user_data_summary_default()
}

fn backup_user_data_default_path() -> Result<String> {
    backup_user_data_default()
}

fn validate_user_data_default_path() -> Result<()> {
    validate_user_data_default()
}

fn sorted_users_by_name_default_path() -> Result<Vec<User>> {
    sorted_users_by_name_default()
}

fn sorted_users_by_id_default_path() -> Result<Vec<User>> {
    sorted_users_by_id_default()
}

fn reactivate_user_default_path(email: &str) -> Result<()> {
    reactivate_user_default(email)
}

fn rename_user_default_path(email: &str, new_name: String) -> Result<()> {
    rename_user_default(email, new_name)
}

fn change_user_email_default_path(email: &str, new_email: String) -> Result<()> {
    change_user_email_default(email, new_email)
}

fn remove_user_default_path(email: &str) -> Result<()> {
    remove_user_default(email)
}

fn clear_user_data_default_path() -> Result<()> {
    clear_user_data_default()
}

fn export_user_data_default_path(out: &str) -> Result<()> {
    export_user_data_default(out)
}

fn import_user_data_default_path(src: &str) -> Result<usize> {
    import_user_data_default(src)
}

fn merge_user_data_default_path(other: &str) -> Result<usize> {
    merge_user_data_default(other)
}

fn restore_user_data_default_path(backup: &str) -> Result<()> {
    restore_user_data_default(backup)
}

fn user_data_exists_default_path() -> bool {
    user_data_exists_default()
}

fn ensure_user_data_default_path() -> Result<()> {
    ensure_user_data_default()
}

fn load_user_data_default_path() -> Result<Vec<User>> {
    load_user_data_default()
}

fn save_user_data_default_path(users: &[User]) -> Result<()> {
    save_user_data_default(users)
}

fn add_user_default_path_2(name: String, email: String) -> Result<User> {
    add_user_default_path(name, email)
}

fn deactivate_user_default_path_2(email: &str) -> Result<()> {
    deactivate_user_default_path(email)
}

fn list_active_users_default_path_2() -> Result<Vec<User>> {
    list_active_users_default_path()
}

fn count_users_default_path_2() -> Result<usize> {
    count_users_default_path()
}

fn search_users_default_path_2(query: &str) -> Result<Vec<User>> {
    search_users_default_path(query)
}

fn user_data_summary_default_path_2() -> Result<String> {
    user_data_summary_default_path()
}

fn backup_user_data_default_path_2() -> Result<String> {
    backup_user_data_default_path()
}

fn validate_user_data_default_path_2() -> Result<()> {
    validate_user_data_default_path()
}

fn sorted_users_by_name_default_path_2() -> Result<Vec<User>> {
    sorted_users_by_name_default_path()
}

fn sorted_users_by_id_default_path_2() -> Result<Vec<User>> {
    sorted_users_by_id_default_path()
}

fn reactivate_user_default_path_2(email: &str) -> Result<()> {
    reactivate_user_default_path(email)
}

fn rename_user_default_path_2(email: &str, new_name: String) -> Result<()> {
    rename_user_default_path(email, new_name)
}

fn change_user_email_default_path_2(email: &str, new_email: String) -> Result<()> {
    change_user_email_default_path(email, new_email)
}

fn remove_user_default_path_2(email: &str) -> Result<()> {
    remove_user_default_path(email)
}

fn clear_user_data_default_path_2() -> Result<()> {
    clear_user_data_default_path()
}

fn export_user_data_default_path_2(out: &str) -> Result<()> {
    export_user_data_default_path(out)
}

fn import_user_data_default_path_2(src: &str) -> Result<usize> {
    import_user_data_default_path(src)
}

fn merge_user_data_default_path_2(other: &str) -> Result<usize> {
    merge_user_data_default_path(other)
}

fn restore_user_data_default_path_2(backup: &str) -> Result<()> {
    restore_user_data_default_path(backup)
}

fn user_data_exists_default_path_2() -> bool {
    user_data_exists_default_path()
}

fn ensure_user_data_default_path_2() -> Result<()> {
    ensure_user_data_default_path()
}

fn load_user_data_default_path_2() -> Result<Vec<User>> {
    load_user_data_default_path()
}

fn save_user_data_default_path_2(users: &[User]) -> Result<()> {
    save_user_data_default_path(users)
}

fn add_user_default_path_3(name: String, email: String) -> Result<User> {
    add_user_default_path_2(name, email)
}

fn deactivate_user_default_path_3(email: &str) -> Result<()> {
    deactivate_user_default_path_2(email)
}

fn list_active_users_default_path_3() -> Result<Vec<User>> {
    list_active_users_default_path_2()
}

fn count_users_default_path_3() -> Result<usize> {
    count_users_default_path_2()
}

fn search_users_default_path_3(query: &str) -> Result<Vec<User>> {
    search_users_default_path_2(query)
}

fn user_data_summary_default_path_3() -> Result<String> {
    user_data_summary_default_path_2()
}

fn backup_user_data_default_path_3() -> Result<String> {
    backup_user_data_default_path_2()
}

fn validate_user_data_default_path_3() -> Result<()> {
    validate_user_data_default_path_2()
}

fn sorted_users_by_name_default_path_3() -> Result<Vec<User>> {
    sorted_users_by_name_default_path_2()
}

fn sorted_users_by_id_default_path_3() -> Result<Vec<User>> {
    sorted_users_by_id_default_path_2()
}

fn reactivate_user_default_path_3(email: &str) -> Result<()> {
    reactivate_user_default_path_2(email)
}

fn rename_user_default_path_3(email: &str, new_name: String) -> Result<()> {
    rename_user_default_path_2(email, new_name)
}

fn change_user_email_default_path_3(email: &str, new_email: String) -> Result<()> {
    change_user_email_default_path_2(email, new_email)
}

fn remove_user_default_path_3(email: &str) -> Result<()> {
    remove_user_default_path_2(email)
}

fn clear_user_data_default_path_3() -> Result<()> {
    clear_user_data_default_path_2()
}

fn export_user_data_default_path_3(out: &str) -> Result<()> {
    export_user_data_default_path_2(out)
}

fn import_user_data_default_path_3(src: &str) -> Result<usize> {
    import_user_data_default_path_2(src)
}

fn merge_user_data_default_path_3(other: &str) -> Result<usize> {
    merge_user_data_default_path_2(other)
}

fn restore_user_data_default_path_3(backup: &str) -> Result<()> {
    restore_user_data_default_path_2(backup)
}

fn user_data_exists_default_path_3() -> bool {
    user_data_exists_default_path_2()
}

fn ensure_user_data_default_path_3() -> Result<()> {
    ensure_user_data_default_path_2()
}

fn load_user_data_default_path_3() -> Result<Vec<User>> {
    load_user_data_default_path_2()
}

fn save_user_data_default_path_3(users: &[User]) -> Result<()> {
    save_user_data_default_path_2(users)
}
```

```
// initialization_error_handler_spec.js

describe('Initialization Error Handler', () => {
  let originalConsoleError;
  let originalWindowOnError;
  let originalWindowOnUnhandledRejection;
  let capturedErrors;
  let errorHandler;

  beforeEach(() => {
    originalConsoleError = console.error;
    originalWindowOnError = window.onerror;
    originalWindowOnUnhandledRejection = window.onunhandledrejection;
    capturedErrors = [];

    console.error = jasmine.createSpy('consoleError');

    errorHandler = new InitializationErrorHandler({
      logToConsole: true,
      logToServer: false,
      maxErrors: 50
    });

    errorHandler.onError((error) => {
      capturedErrors.push(error);
    });
  });

  afterEach(() => {
    errorHandler.uninstall();
    console.error = originalConsoleError;
    window.onerror = originalWindowOnError;
    window.onunhandledrejection = originalWindowOnUnhandledRejection;
  });

  describe('installation', () => {
    it('should install global error handlers', () => {
      errorHandler.install();
      expect(window.onerror).not.toBeNull();
      expect(window.onunhandledrejection).not.toBeNull();
    });

    it('should not install twice', () => {
      errorHandler.install();
      const firstHandler = window.onerror;
      errorHandler.install();
      expect(window.onerror).toBe(firstHandler);
    });

    it('should restore original handlers on uninstall', () => {
      errorHandler.install();
      errorHandler.uninstall();
      expect(window.onerror).toBe(originalWindowOnError);
      expect(window.onunhandledrejection).toBe(originalWindowOnUnhandledRejection);
    });
  });

  describe('error capturing', () => {
    it('should capture synchronous errors', () => {
      errorHandler.install();
      window.onerror('Test error', 'test.js', 10, 5, new Error('Test error'));
      expect(capturedErrors.length).toBe(1);
      expect(capturedErrors[0].message).toBe('Test error');
      expect(capturedErrors[0].source).toBe('test.js');
      expect(capturedErrors[0].line).toBe(10);
      expect(capturedErrors[0].column).toBe(5);
    });

    it('should capture unhandled promise rejections', () => {
      errorHandler.install();
      const event = {
        reason: new Error('Promise rejection'),
        promise: Promise.resolve()
      };
      window.onunhandledrejection(event);
      expect(capturedErrors.length).toBe(1);
      expect(capturedErrors[0].message).toBe('Promise rejection');
      expect(capturedErrors[0].type).toBe('unhandledrejection');
    });

    it('should capture string errors', () => {
      errorHandler.install();
      window.onerror('String error', 'test.js', 1, 1, null);
      expect(capturedErrors.length).toBe(1);
      expect(capturedErrors[0].message).toBe('String error');
    });

    it('should capture errors with stack traces', () => {
      errorHandler.install();
      const error = new Error('Stack test');
      window.onerror('Stack test', 'test.js', 1, 1, error);
      expect(capturedErrors[0].stack).toBeDefined();
      expect(capturedErrors[0].stack).toContain('Stack test');
    });
  });

  describe('error categorization', () => {
    it('should categorize network errors', () => {
      errorHandler.install();
      window.onerror('Failed to fetch', 'test.js', 1, 1, new Error('Failed to fetch'));
      expect(capturedErrors[0].category).toBe('network');
    });

    it('should categorize timeout errors', () => {
      errorHandler.install();
      window.onerror('Request timeout', 'test.js', 1, 1, new Error('Request timeout'));
      expect(capturedErrors[0].category).toBe('timeout');
    });

    it('should categorize permission errors', () => {
      errorHandler.install();
      window.onerror('Permission denied', 'test.js', 1, 1, new Error('Permission denied'));
      expect(capturedErrors[0].category).toBe('permission');
    });

    it('should categorize resource errors', () => {
      errorHandler.install();
      window.onerror('Failed to load resource', 'test.js', 1, 1, new Error('Failed to load resource'));
      expect(capturedErrors[0].category).toBe('resource');
    });

    it('should categorize unknown errors', () => {
      errorHandler.install();
      window.onerror('Something went wrong', 'test.js', 1, 1, new Error('Something went wrong'));
      expect(capturedErrors[0].category).toBe('unknown');
    });
  });

  describe('error limits', () => {
    it('should respect maxErrors limit', () => {
      errorHandler.install();
      for (let i = 0; i < 60; i++) {
        window.onerror(`Error ${i}`, 'test.js', i, 1, new Error(`Error ${i}`));
      }
      expect(capturedErrors.length).toBe(50);
    });

    it('should keep most recent errors when limit exceeded', () => {
      errorHandler.install();
      for (let i = 0; i < 60; i++) {
        window.onerror(`Error ${i}`, 'test.js', i, 1, new Error(`Error ${i}`));
      }
      expect(capturedErrors[0].message).toBe('Error 10');
      expect(capturedErrors[49].message).toBe('Error 59');
    });
  });

  describe('error clearing', () => {
    it('should clear all errors', () => {
      errorHandler.install();
      window.onerror('Error 1', 'test.js', 1, 1, new Error('Error 1'));
      window.onerror('Error 2', 'test.js', 2, 1, new Error('Error 2'));
      errorHandler.clearErrors();
      expect(errorHandler.getErrors().length).toBe(0);
    });
  });

  describe('error retrieval', () => {
    it('should return all errors', () => {
      errorHandler.install();
      window.onerror('Error 1', 'test.js', 1, 1, new Error('Error 1'));
      window.onerror('Error 2', 'test.js', 2, 1, new Error('Error 2'));
      const errors = errorHandler.getErrors();
      expect(errors.length).toBe(2);
    });

    it('should return errors by category', () => {
      errorHandler.install();
      window.onerror('Failed to fetch', 'test.js', 1, 1, new Error('Failed to fetch'));
      window.onerror('Permission denied', 'test.js', 2, 1, new Error('Permission denied'));
      const networkErrors = errorHandler.getErrorsByCategory('network');
      expect(networkErrors.length).toBe(1);
      expect(networkErrors[0].message).toBe('Failed to fetch');
    });
  });

  describe('error reporting', () => {
    it('should generate error report', () => {
      errorHandler.install();
      window.onerror('Error 1', 'test.js', 1, 1, new Error('Error 1'));
      const report = errorHandler.generateReport();
      expect(report.totalErrors).toBe(1);
      expect(report.errors).toBeDefined();
      expect(report.timestamp).toBeDefined();
    });
  });
});

// network_request_interceptor_spec.js

describe('Network Request Interceptor', () => {
  let originalFetch;
  let originalXHROpen;
  let originalXHRSend;
  let interceptedRequests;
  let interceptor;

  beforeEach(() => {
    originalFetch = window.fetch;
    originalXHROpen = XMLHttpRequest.prototype.open;
    originalXHRSend = XMLHttpRequest.prototype.send;
    interceptedRequests = [];

    interceptor = new NetworkRequestInterceptor({
      interceptFetch: true,
      interceptXHR: true,
      maxRequests: 100
    });

    interceptor.onRequest((request) => {
      interceptedRequests.push(request);
    });
  });

  afterEach(() => {
    interceptor.uninstall();
    window.fetch = originalFetch;
    XMLHttpRequest.prototype.open = originalXHROpen;
    XMLHttpRequest.prototype.send = originalXHRSend;
  });

  describe('installation', () => {
    it('should install fetch interceptor', () => {
      interceptor.install();
      expect(window.fetch).not.toBe(originalFetch);
    });

    it('should install XHR interceptor', () => {
      interceptor.install();
      expect(XMLHttpRequest.prototype.open).not.toBe(originalXHROpen);
      expect(XMLHttpRequest.prototype.send).not.toBe(originalXHRSend);
    });

    it('should not install twice', () => {
      interceptor.install();
      const firstFetch = window.fetch;
      interceptor.install();
      expect(window.fetch).toBe(firstFetch);
    });

    it('should restore original functions on uninstall', () => {
      interceptor.install();
      interceptor.uninstall();
      expect(window.fetch).toBe(originalFetch);
      expect(XMLHttpRequest.prototype.open).toBe(originalXHROpen);
      expect(XMLHttpRequest.prototype.send).toBe(originalXHRSend);
    });
  });

  describe('fetch interception', () => {
    it('should intercept fetch requests', (done) => {
      interceptor.install();
      fetch('/api/test').then(() => {
        expect(interceptedRequests.length).toBe(1);
        expect(interceptedRequests[0].url).toBe('/api/test');
        expect(interceptedRequests[0].method).toBe('GET');
        done();
      });
    });

    it('should capture fetch request method', (done) => {
      interceptor.install();
      fetch('/api/test', { method: 'POST' }).then(() => {
        expect(interceptedRequests[0].method).toBe('POST');
        done();
      });
    });

    it('should capture fetch request headers', (done) => {
      interceptor.install();
      fetch('/api/test', { headers: { 'X-Custom': 'value' } }).then(() => {
        expect(interceptedRequests[0].headers['X-Custom']).toBe('value');
        done();
      });
    });

    it('should capture fetch response status', (done) => {
      interceptor.install();
      fetch('/api/test').then(() => {
        expect(interceptedRequests[0].status).toBeDefined();
        done();
      });
    });

    it('should capture fetch duration', (done) => {
      interceptor.install();
      fetch('/api/test').then(() => {
        expect(interceptedRequests[0].duration).toBeGreaterThan(0);
        done();
      });
    });
  });

  describe('XHR interception', () => {
    it('should intercept XHR requests', (done) => {
      interceptor.install();
      const xhr = new XMLHttpRequest();
      xhr.open('GET', '/api/test');
      xhr.send();
      xhr.onload = () => {
        expect(interceptedRequests.length).toBe(1);
        expect(interceptedRequests[0].url).toBe('/api/test');
        expect(interceptedRequests[0].method).toBe('GET');
        done();
      };
    });

    it('should capture XHR request method', (done) => {
      interceptor.install();
      const xhr = new XMLHttpRequest();
      xhr.open('POST', '/api/test');
      xhr.send();
      xhr.onload = () => {
        expect(interceptedRequests[0].method).toBe('POST');
        done();
      };
    });

    it('should capture XHR request headers', (done) => {
      interceptor.install();
      const xhr = new XMLHttpRequest();
      xhr.open('GET', '/api/test');
      xhr.setRequestHeader('X-Custom', 'value');
      xhr.send();
      xhr.onload = () => {
        expect(interceptedRequests[0].headers['X-Custom']).toBe('value');
        done();
      };
    });

    it('should capture XHR response status', (done) => {
      interceptor.install();
      const xhr = new XMLHttpRequest();
      xhr.open('GET', '/api/test');
      xhr.send();
      xhr.onload = () => {
        expect(interceptedRequests[0].status).toBeDefined();
        done();
      };
    });

    it('should capture XHR duration', (done) => {
      interceptor.install();
      const xhr = new XMLHttpRequest();
      xhr.open('GET', '/api/test');
      xhr.send();
      xhr.onload = () => {
        expect(interceptedRequests[0].duration).toBeGreaterThan(0);
        done();
      };
    });
  });

  describe('request limits', () => {
    it('should respect maxRequests limit', (done) => {
      interceptor.install();
      let completed = 0;
      for (let i = 0; i < 110; i++) {
        fetch(`/api/test${i}`).then(() => {
          completed++;
          if (completed === 110) {
            expect(interceptedRequests.length).toBe(100);
            done();
          }
        });
      }
    });

    it('should keep most recent requests when limit exceeded', (done) => {
      interceptor.install();
      let completed = 0;
      for (let i = 0; i < 110; i++) {
        fetch(`/api/test${i}`).then(() => {
          completed++;
          if (completed === 110) {
            expect(interceptedRequests[0].url).toBe('/api/test10');
            expect(interceptedRequests[99].url).toBe('/api/test109');
            done();
          }
        });
      }
    });
  });

  describe('request clearing', () => {
    it('should clear all requests', (done) => {
      interceptor.install();
      fetch('/api/test1').then(() => {
        fetch('/api/test2').then(() => {
          interceptor.clearRequests();
          expect(interceptor.getRequests().length).toBe(0);
          done();
        });
      });
    });
  });

  describe('request retrieval', () => {
    it('should return all requests', (done) => {
      interceptor.install();
      fetch('/api/test1').then(() => {
        fetch('/api/test2').then(() => {
          const requests = interceptor.getRequests();
          expect(requests.length).toBe(2);
          done();
        });
      });
    });

    it('should return requests by method', (done) => {
      interceptor.install();
      fetch('/api/test1', { method: 'GET' }).then(() => {
        fetch('/api/test2', { method: 'POST' }).then(() => {
          const getRequests = interceptor.getRequestsByMethod('GET');
          expect(getRequests.length).toBe(1);
          expect(getRequests[0].method).toBe('GET');
          done();
        });
      });
    });

    it('should return requests by URL pattern', (done) => {
      interceptor.install();
      fetch('/api/users').then(() => {
        fetch('/api/posts').then(() => {
          const userRequests = interceptor.getRequestsByUrl('/api/users');
          expect(userRequests.length).toBe(1);
          expect(userRequests[0].url).toBe('/api/users');
          done();
        });
      });
    });
  });

  describe('request reporting', () => {
    it('should generate request report', (done) => {
      interceptor.install();
      fetch('/api/test').then(() => {
        const report = interceptor.generateReport();
        expect(report.totalRequests).toBe(1);
        expect(report.requests).toBeDefined();
        expect(report.timestamp).toBeDefined();
        done();
      });
    });
  });
});

// memory_leak_detector_spec.js

describe('Memory Leak Detector', () => {
  let detector;
  let leakedObjects;

  beforeEach(() => {
    leakedObjects = [];
    detector = new MemoryLeakDetector({
      checkInterval: 1000,
      threshold: 10,
      maxSnapshots: 20
    });
  });

  afterEach(() => {
    detector.stop();
  });

  describe('initialization', () => {
    it('should create detector with default options', () => {
      const defaultDetector = new MemoryLeakDetector();
      expect(defaultDetector.checkInterval).toBeDefined();
      expect(defaultDetector.threshold).toBeDefined();
      expect(defaultDetector.maxSnapshots).toBeDefined();
    });

    it('should create detector with custom options', () => {
      expect(detector.checkInterval).toBe(1000);
      expect(detector.threshold).toBe(10);
      expect(detector.maxSnapshots).toBe(20);
    });

    it('should not be running initially', () => {
      expect(detector.isRunning()).toBe(false);
    });
  });

  describe('start and stop', () => {
    it('should start monitoring', () => {
      detector.start();
      expect(detector.isRunning()).toBe(true);
    });

    it('should stop monitoring', () => {
      detector.start();
      detector.stop();
      expect(detector.isRunning()).toBe(false);
    });

    it('should not start twice', () => {
      detector.start();
      const firstInterval = detector.intervalId;
      detector.start();
      expect(detector.intervalId).toBe(firstInterval);
    });
  });

  describe('snapshot taking', () => {
    it('should take a snapshot', () => {
      detector.takeSnapshot();
      expect(detector.snapshots.length).toBe(1);
    });

    it('should record snapshot timestamp', () => {
      const before = Date.now();
      detector.takeSnapshot();
      const after = Date.now();
      const snapshot = detector.snapshots[0];
      expect(snapshot.timestamp).toBeGreaterThanOrEqual(before);
      expect(snapshot.timestamp).toBeLessThanOrEqual(after);
    });

    it('should record memory usage', () => {
      detector.takeSnapshot();
      const snapshot = detector.snapshots[0];
      expect(snapshot.usedJSHeapSize).toBeGreaterThan(0);
      expect(snapshot.totalJSHeapSize).toBeGreaterThan(0);
    });

    it('should respect maxSnapshots limit', () => {
      for (let i = 0; i < 30; i++) {
        detector.takeSnapshot();
      }
      expect(detector.snapshots.length).toBe(20);
    });

    it('should keep most recent snapshots when limit exceeded', () => {
      for (let i = 0; i < 30; i++) {
        detector.takeSnapshot();
      }
      const firstSnapshot = detector.snapshots[0];
      const lastSnapshot = detector.snapshots[19];
      expect(lastSnapshot.timestamp).toBeGreaterThanOrEqual(firstSnapshot.timestamp);
    });
  });

  describe('leak detection', () => {
    it('should detect memory leak when threshold exceeded', () => {
      const leaks = [];
      detector.onLeakDetected((leak) => {
        leaks.push(leak);
      });
      for (let i = 0; i < 15; i++) {
        leakedObjects.push(new Array(1000000));
        detector.takeSnapshot();
      }
      detector.checkForLeaks();
      expect(leaks.length).toBeGreaterThan(0);
    });

    it('should not detect leak when memory is stable', () => {
      const leaks = [];
      detector.onLeakDetected((leak) => {
        leaks.push(leak);
      });
      for (let i = 0; i < 15; i++) {
        detector.takeSnapshot();
      }
      detector.checkForLeaks();
      expect(leaks.length).toBe(0);
    });

    it('should calculate memory growth rate', () => {
      for (let i = 0; i < 5; i++) {
        leakedObjects.push(new Array(1000000));
        detector.takeSnapshot();
      }
      const growthRate = detector.calculateGrowthRate();
      expect(growthRate).toBeGreaterThan(0);
    });
  });

  describe('snapshot clearing', () => {
    it('should clear all snapshots', () => {
      detector.takeSnapshot();
      detector.takeSnapshot();
      detector.clearSnapshots();
      expect(detector.snapshots.length).toBe(0);
    });
  });

  describe('snapshot retrieval', () => {
    it('should return all snapshots', () => {
      detector.takeSnapshot();
      detector.takeSnapshot();
      const snapshots = detector.getSnapshots();
      expect(snapshots.length).toBe(2);
    });

    it('should return snapshots in chronological order', () => {
      detector.takeSnapshot();
      detector.takeSnapshot();
      const snapshots = detector.getSnapshots();
      expect(snapshots[0].timestamp).toBeLessThanOrEqual(snapshots[1].timestamp);
    });
  });

  describe('report generation', () => {
    it('should generate leak report', () => {
      detector.takeSnapshot();
      const report = detector.generateReport();
      expect(report.snapshots).toBeDefined();
      expect(report.timestamp).toBeDefined();
      expect(report.leaksDetected).toBeDefined();
    });
  });
});

// dom_mutation_observer_spec.js

describe('DOM Mutation Observer', () => {
  let observer;
  let mutations;
  let container;

  beforeEach(() => {
    mutations = [];
    container = document.createElement('div');
    document.body.appendChild(container);

    observer = new DOMMutationObserver(container, {
      childList: true,
      attributes: true,
      characterData: true,
      subtree: true
    });

    observer.onMutation((mutation) => {
      mutations.push(mutation);
    });
  });

  afterEach(() => {
    observer.disconnect();
    document.body.removeChild(container);
  });

  describe('initialization', () => {
    it('should create observer', () => {
      expect(observer).toBeDefined();
    });

    it('should not be observing initially', () => {
      expect(observer.isObserving()).toBe(false);
    });
  });

  describe('observing', () => {
    it('should start observing', () => {
      observer.observe();
      expect(observer.isObserving()).toBe(true);
    });

    it('should stop observing', () => {
      observer.observe();
      observer.disconnect();
      expect(observer.isObserving()).toBe(false);
    });
  });

  describe('child list mutations', () => {
    it('should detect added children', (done) => {
      observer.observe();
      const child = document.createElement('span');
      container.appendChild(child);
      setTimeout(() => {
        expect(mutations.length).toBeGreaterThan(0);
        expect(mutations[0].type).toBe('childList');
        expect(mutations[0].addedNodes.length).toBe(1);
        done();
      }, 100);
    });

    it('should detect removed children', (done) => {
      const child = document.createElement('span');
      container.appendChild(child);
      observer.observe();
      container.removeChild(child);
      setTimeout(() => {
        expect(mutations.length).toBeGreaterThan(0);
        expect(mutations[0].type).toBe('childList');
        expect(mutations[0].removedNodes.length).toBe(1);
        done();
      }, 100);
    });
  });

  describe('attribute mutations', () => {
    it('should detect attribute changes', (done) => {
      observer.observe();
      container.setAttribute('data-test', 'value');
      setTimeout(() => {
        expect(mutations.length).toBeGreaterThan(0);
        expect(mutations[0].type).toBe('attributes');
        expect(mutations[0].attributeName).toBe('data-test');
        done();
      }, 100);
    });

    it('should detect old attribute value', (done) => {
      container.setAttribute('data-test', 'old');
      observer.observe();
      container.setAttribute('data-test', 'new');
      setTimeout(() => {
        expect(mutations[0].oldValue).toBe('old');
        done();
      }, 100);
    });
  });

  describe('character data mutations', () => {
    it('should detect text changes', (done) => {
      const textNode = document.createTextNode('old');
      container.appendChild(textNode);
      observer.observe();
      textNode.textContent = 'new';
      setTimeout(() => {
        expect(mutations.length).toBeGreaterThan(0);
        expect(mutations[0].type).toBe('characterData');
        done();
      }, 100);
    });
  });

  describe('mutation clearing', () => {
    it('should clear all mutations', (done) => {
      observer.observe();
      container.appendChild(document.createElement('span'));
      setTimeout(() => {
        observer.clearMutations();
        expect(observer.getMutations().length).toBe(0);
        done();
      }, 100);
    });
  });

  describe('mutation retrieval', () => {
    it('should return all mutations', (done) => {
      observer.observe();
      container.appendChild(document.createElement('span'));
      container.setAttribute('data-test', 'value');
      setTimeout(() => {
        const allMutations = observer.getMutations();
        expect(allMutations.length).toBeGreaterThan(1);
        done();
      }, 100);
    });

    it('should return mutations by type', (done) => {
      observer.observe();
      container.appendChild(document.createElement('span'));
      container.setAttribute('data-test', 'value');
      setTimeout(() => {
        const childListMutations = observer.getMutationsByType('childList');
        expect(childListMutations.length).toBeGreaterThan(0);
        expect(childListMutations[0].type).toBe('childList');
        done();
      }, 100);
    });
  });
});

// performance_profiler_spec.js

describe('Performance Profiler', () => {
  let profiler;
  let profiles;

  beforeEach(() => {
    profiles = [];
    profiler = new PerformanceProfiler({
      sampleInterval: 10,
      maxProfiles: 10
    });
  });

  afterEach(() => {
    profiler.stop();
  });

  describe('initialization', () => {
    it('should create profiler with default options', () => {
      const defaultProfiler = new PerformanceProfiler();
      expect(defaultProfiler.sampleInterval).toBeDefined();
      expect(defaultProfiler.maxProfiles).toBeDefined();
    });

    it('should create profiler with custom options', () => {
      expect(profiler.sampleInterval).toBe(10);
      expect(profiler.maxProfiles).toBe(10);
    });

    it('should not be profiling initially', () => {
      expect(profiler.isProfiling()).toBe(false);
    });
  });

  describe('start and stop', () => {
    it('should start profiling', () => {
      profiler.start('test');
      expect(profiler.isProfiling()).toBe(true);
    });

    it('should stop profiling', () => {
      profiler.start('test');
      profiler.stop();
      expect(profiler.isProfiling()).toBe(false);
    });

    it('should not start twice', () => {
      profiler.start('test');
      const firstProfile = profiler.currentProfile;
      profiler.start('test2');
      expect(profiler.currentProfile).toBe(firstProfile);
    });
  });

  describe('profile recording', () => {
    it('should record profile', (done) => {
      profiler.start('test');
      setTimeout(() => {
        profiler.stop();
        expect(profiler.profiles.length).toBe(1);
        done();
      }, 100);
    });

    it('should record profile name', (done) => {
      profiler.start('test');
      setTimeout(() => {
        profiler.stop();
        expect(profiler.profiles[0].name).toBe('test');
        done();
      }, 100);
    });

    it('should record profile duration', (done) => {
      profiler.start('test');
      setTimeout(() => {
        profiler.stop();
        expect(profiler.profiles[0].duration).toBeGreaterThan(0);
        done();
      }, 100);
    });

    it('should record samples', (done) => {
      profiler.start('test');
      setTimeout(() => {
        profiler.stop();
        expect(profiler.profiles[0].samples.length).toBeGreaterThan(0);
        done();
      }, 100);
    });

    it('should respect maxProfiles limit', (done) => {
      let completed = 0;
      for (let i = 0; i < 15; i++) {
        profiler.start(`test${i}`);
        setTimeout(() => {
          profiler.stop();
          completed++;
          if (completed === 15) {
            expect(profiler.profiles.length).toBe(10);
            done();
          }
        }, 10);
      }
    });
  });

  describe('profile clearing', () => {
    it('should clear all profiles', (done) => {
      profiler.start('test');
      setTimeout(() => {
        profiler.stop();
        profiler.clearProfiles();
        expect(profiler.profiles.length).toBe(0);
        done();
      }, 100);
    });
  });

  describe('profile retrieval', () => {
    it('should return all profiles', (done) => {
      profiler.start('test1');
      setTimeout(() => {
        profiler.stop();
        profiler.start('test2');
        setTimeout(() => {
          profiler.stop();
          const profiles = profiler.getProfiles();
          expect(profiles.length).toBe(2);
          done();
        }, 100);
      }, 100);
    });

    it('should return profile by name', (done) => {
      profiler.start('test1');
      setTimeout(() => {
        profiler.stop();
        profiler.start('test2');
        setTimeout(() => {
          profiler.stop();
          const profile = profiler.getProfileByName('test1');
          expect(profile.name).toBe('test1');
          done();
        }, 100);
      }, 100);
    });
  });

  describe('report generation', () => {
    it('should generate profile report', (done) => {
      profiler.start('test');
      setTimeout(() => {
        profiler.stop();
        const report = profiler.generateReport();
        expect(report.profiles).toBeDefined();
        expect(report.timestamp).toBeDefined();
        done();
      }, 100);
    });
  });
});

// resource_timing_collector_spec.js

describe('Resource Timing Collector', () => {
  let collector;
  let resources;

  beforeEach(() => {
    resources = [];
    collector = new ResourceTimingCollector({
      maxResources: 100
    });
  });

  afterEach(() => {
    collector.stop();
  });

  describe('initialization', () => {
    it('should create collector with default options', () => {
      const defaultCollector = new ResourceTimingCollector();
      expect(defaultCollector.maxResources).toBeDefined();
    });

    it('should create collector with custom options', () => {
      expect(collector.maxResources).toBe(100);
    });

    it('should not be collecting initially', () => {
      expect(collector.isCollecting()).toBe(false);
    });
  });

  describe('start and stop', () => {
    it('should start collecting', () => {
      collector.start();
      expect(collector.isCollecting()).toBe(true);
    });

    it('should stop collecting', () => {
      collector.start();
      collector.stop();
      expect(collector.isCollecting()).toBe(false);
    });
  });

  describe('resource collection', () => {
    it('should collect resource timings', (done) => {
      collector.start();
      fetch('/api/test').then(() => {
        setTimeout(() => {
          expect(collector.resources.length).toBeGreaterThan(0);
          done();
        }, 100);
      });
    });

    it('should record resource name', (done) => {
      collector.start();
      fetch('/api/test').then(() => {
        setTimeout(() => {
          expect(collector.resources[0].name).toContain('/api/test');
          done();
        }, 100);
      });
    });

    it('should record resource duration', (done) => {
      collector.start();
      fetch('/api/test').then(() => {
        setTimeout(() => {
          expect(collector.resources[0].duration).toBeGreaterThan(0);
          done();
        }, 100);
      });
    });

    it('should respect maxResources limit', (done) => {
      collector.start();
      let completed = 0;
      for (let i = 0; i < 110; i++) {
        fetch(`/api/test${i}`).then(() => {
          completed++;
          if (completed === 110) {
            setTimeout(() => {
              expect(collector.resources.length).toBe(100);
              done();
            }, 100);
          }
        });
      }
    });
  });

  describe('resource clearing', () => {
    it('should clear all resources', (done) => {
      collector.start();
      fetch('/api/test').then(() => {
        setTimeout(() => {
          collector.clearResources();
          expect(collector.getResources().length).toBe(0);
          done();
        }, 100);
      });
    });
  });

  describe('resource retrieval', () => {
    it('should return all resources', (done) => {
      collector.start();
      fetch('/api/test1').then(() => {
        fetch('/api/test2').then(() => {
          setTimeout(() => {
            const resources = collector.getResources();
            expect(resources.length).toBe(2);
            done();
          }, 100);
        });
      });
    });

    it('should return resources by type', (done) => {
      collector.start();
      fetch('/api/test').then(() => {
        setTimeout(() => {
          const fetchResources = collector.getResourcesByType('fetch');
          expect(fetchResources.length).toBeGreaterThan(0);
          done();
        }, 100);
      });
    });
  });

  describe('report generation', () => {
    it('should generate resource report', (done) => {
      collector.start();
      fetch('/api/test').then(() => {
        setTimeout(() => {
          const report = collector.generateReport();
          expect(report.resources).toBeDefined();
          expect(report.timestamp).toBeDefined();
          done();
        }, 100);
      });
    });
  });
});

// user_timing_collector_spec.js

describe('User Timing Collector', () => {
  let collector;
  let measures;

  beforeEach(() => {
    measures = [];
    collector = new UserTimingCollector({
      maxMeasures: 100
    });
  });

  afterEach(() => {
    collector.stop();
  });

  describe('initialization', () => {
    it('should create collector with default options', () => {
      const defaultCollector = new UserTimingCollector();
      expect(defaultCollector.maxMeasures).toBeDefined();
    });

    it('should create collector with custom options', () => {
      expect(collector.maxMeasures).toBe(100);
    });

    it('should not be collecting initially', () => {
      expect(collector.isCollecting()).toBe(false);
    });
  });

  describe('start and stop', () => {
    it('should start collecting', () => {
      collector.start();
      expect(collector.isCollecting()).toBe(true);
    });

    it('should stop collecting', () => {
      collector.start();
      collector.stop();
      expect(collector.isCollecting()).toBe(false);
    });
  });

  describe('measure collection', () => {
    it('should collect user timings', () => {
      collector.start();
      performance.mark('test-start');
      performance.mark('test-end');
      performance.measure('test', 'test-start', 'test-end');
      expect(collector.measures.length).toBeGreaterThan(0);
    });

    it('should record measure name', () => {
      collector.start();
      performance.mark('test-start');
      performance.mark('test-end');
      performance.measure('test', 'test-start', 'test-end');
      expect(collector.measures[0].name).toBe('test');
    });

    it('should record measure duration', () => {
      collector.start();
      performance.mark('test-start');
      performance.mark('test-end');
      performance.measure('test', 'test-start', 'test-end');
      expect(collector.measures[0].duration).toBeGreaterThanOrEqual(0);
    });

    it('should respect maxMeasures limit', () => {
      collector.start();
      for (let i = 0; i < 110; i++) {
        performance.mark(`test-start-${i}`);
        performance.mark(`test-end-${i}`);
        performance.measure(`test${i}`, `test-start-${i}`, `test-end-${i}`);
      }
      expect(collector.measures.length).toBe(100);
    });
  });

  describe('measure clearing', () => {
    it('should clear all measures', () => {
      collector.start();
      performance.mark('test-start');
      performance.mark('test-end');
      performance.measure('test', 'test-start', 'test-end');
      collector.clearMeasures();
      expect(collector.getMeasures().length).toBe(0);
    });
  });

  describe('measure retrieval', () => {
    it('should return all measures', () => {
      collector.start();
      performance.mark('test1-start');
      performance.mark('test1-end');
      performance.measure('test1', 'test1-start', 'test1-end');
      performance.mark('test2-start');
      performance.mark('test2-end');
      performance.measure('test2', 'test2-start', 'test2-end');
      const measures = collector.getMeasures();
      expect(measures.length).toBe(2);
    });

    it('should return measures by name', () => {
      collector.start();
      performance.mark('test1-start');
      performance.mark('test1-end');
      performance.measure('test1', 'test1-start', 'test1-end');
      performance.mark('test2-start');
      performance.mark('test2-end');
      performance.measure('test2', 'test2-start', 'test2-end');
      const measure = collector.getMeasureByName('test1');
      expect(measure.name).toBe('test1');
    });
  });

  describe('report generation', () => {
    it('should generate measure report', () => {
      collector.start();
      performance.mark('test-start');
      performance.mark('test-end');
      performance.measure('test', 'test-start', 'test-end');
      const report = collector.generateReport();
      expect(report.measures).toBeDefined();
      expect(report.timestamp).toBeDefined();
    });
  });
});

// long_task_detector_spec.js

describe('Long Task Detector', () => {
  let detector;
  let longTasks;

  beforeEach(() => {
    longTasks = [];
    detector = new LongTaskDetector({
      threshold: 50,
      maxTasks: 100
    });

    detector.onLongTask((task) => {
      longTasks.push(task);
    });
  });

  afterEach(() => {
    detector.stop();
  });

  describe('initialization', () => {
    it('should create detector with default options', () => {
      const defaultDetector = new LongTaskDetector();
      expect(defaultDetector.threshold).toBeDefined();
      expect(defaultDetector.maxTasks).toBeDefined();
    });

    it('should create detector with custom options', () => {
      expect(detector.threshold).toBe(50);
      expect(detector.maxTasks).toBe(100);
    });

    it('should not be detecting initially', () => {
      expect(detector.isDetecting()).toBe(false);
    });
  });

  describe('start and stop', () => {
    it('should start detecting', () => {
      detector.start();
      expect(detector.isDetecting()).toBe(true);
    });

    it('should stop detecting', () => {
      detector.start();
      detector.stop();
      expect(detector.isDetecting()).toBe(false);
    });
  });

  describe('long task detection', () => {
    it('should detect long tasks', (done) => {
      detector.start();
      const start = Date.now();
      while (Date.now() - start < 100) {
        // Busy wait
      }
      setTimeout(() => {
        expect(longTasks.length).toBeGreaterThan(0);
        done();
      }, 100);
    });

    it('should record task duration', (done) => {
      detector.start();
      const start = Date.now();
      while (Date.now() - start < 100) {
        // Busy wait
      }
      setTimeout(() => {
        expect(longTasks[0].duration).toBeGreaterThan(50);
        done();
      }, 100);
    });

    it('should record task start time', (done) => {
      detector.start();
      const start = Date.now();
      while (Date.now() - start < 100) {
        // Busy wait
      }
      setTimeout(() => {
        expect(longTasks[0].startTime).toBeGreaterThan(0);
        done();
      }, 100);
    });

    it('should respect maxTasks limit', (done) => {
      detector.start();
      for (let i = 0; i < 110; i++) {
        const start = Date.now();
        while (Date.now() - start < 60) {
          // Busy wait
        }
      }
      setTimeout(() => {
        expect(longTasks.length).toBe(100);
        done();
      }, 100);
    });
  });

  describe('task clearing', () => {
    it('should clear all tasks', (done) => {
      detector.start();
      const start = Date.now();
      while (Date.now() - start < 100) {
        // Busy wait
      }
      setTimeout(() => {
        detector.clearTasks();
        expect(detector.getTasks().length).toBe(0);
        done();
      }, 100);
    });
  });

  describe('task retrieval', () => {
    it('should return all tasks', (done) => {
      detector.start();
      const start = Date.now();
      while (Date.now() - start < 100) {
        // Busy wait
      }
      setTimeout(() => {
        const tasks = detector.getTasks();
        expect(tasks.length).toBeGreaterThan(0);
        done();
      }, 100);
    });
  });

  describe('report generation', () => {
    it('should generate task report', (done) => {
      detector.start();
      const start = Date.now();
      while (Date.now() - start < 100) {
        // Busy wait
      }
      setTimeout(() => {
        const report = detector.generateReport();
        expect(report.tasks).toBeDefined();
        expect(report.timestamp).toBeDefined();
        done();
      }, 100);
    });
  });
});

// frame_rate_monitor_spec.js

describe('Frame Rate Monitor', () => {
  let monitor;
  let frameRates;

  beforeEach(() => {
    frameRates = [];
    monitor = new FrameRateMonitor({
      sampleSize: 60,
      threshold: 30
    });

    monitor.onFrameRate((rate) => {
      frameRates.push(rate);
    });
  });

  afterEach(() => {
    monitor.stop();
  });

  describe('initialization', () => {
    it('should create monitor with default options', () => {
      const defaultMonitor = new FrameRateMonitor();
      expect(defaultMonitor.sampleSize).toBeDefined();
      expect(defaultMonitor.threshold).toBeDefined();
    });

    it('should create monitor with custom options', () => {
      expect(monitor.sampleSize).toBe(60);
      expect(monitor.threshold).toBe(30);
    });

    it('should not be monitoring initially', () => {
      expect(monitor.isMonitoring()).toBe(false);
    });
  });

  describe('start and stop', () => {
    it('should start monitoring', () => {
      monitor.start();
      expect(monitor.isMonitoring()).toBe(true);
    });

    it('should stop monitoring', () => {
      monitor.start();
      monitor.stop();
      expect(monitor.isMonitoring()).toBe(false);
    });
  });

  describe('frame rate measurement', () => {
    it('should measure frame rate', (done) => {
      monitor.start();
      setTimeout(() => {
        expect(frameRates.length).toBeGreaterThan(0);
        done();
      }, 1000);
    });

    it('should calculate average frame rate', (done) => {
      monitor.start();
      setTimeout(() => {
        const avgRate = monitor.getAverageFrameRate();
        expect(avgRate).toBeGreaterThan(0);
        done();
      }, 1000);
    });

    it('should detect low frame rate', (done) => {
      monitor.start();
      const start = Date.now();
      while (Date.now() - start < 1000) {
        // Busy wait to cause low frame rate
      }
      setTimeout(() => {
        const lowFrameRates = frameRates.filter(rate => rate < 30);
        expect(lowFrameRates.length).toBeGreaterThan(0);
        done();
      }, 1000);
    });
  });

  describe('report generation', () => {
    it('should generate frame rate report', (done) => {
      monitor.start();
      setTimeout(() => {
        const report = monitor.generateReport();
        expect(report.frameRates).toBeDefined();
        expect(report.timestamp).toBeDefined();
        done();
      }, 1000);
    });
  });
});

// battery_status_monitor_spec.js

describe('Battery Status Monitor', () => {
  let monitor;
  let batteryStatuses;

  beforeEach(() => {
    batteryStatuses = [];
    monitor = new BatteryStatusMonitor({
      checkInterval: 1000
    });

    monitor.onBatteryStatus((status) => {
      batteryStatuses.push(status);
    });
  });

  afterEach(() => {
    monitor.stop();
  });

  describe('initialization', () => {
    it('should create monitor with default options', () => {
      const defaultMonitor = new BatteryStatusMonitor();
      expect(defaultMonitor.checkInterval).toBeDefined();
    });

    it('should create monitor with custom options', () => {
      expect(monitor.checkInterval).toBe(1000);
    });

    it('should not be monitoring initially', () => {
      expect(monitor.isMonitoring()).toBe(false);
    });
  });

  describe('start
```